부록 · Docker 샌드박스 · Compose

Docker Compose 샌드박스 20가지 개발 격리 검증 교본

Docker Compose 샌드박스 20가지 개발 및 격리 검증 실전 교본

Codex CLI · SSH · MariaDB · Kotlin/Spring Boot · Rust · Redis · Ollama · Kafka/Redpanda · Traefik · Prometheus

본 문서는 개발 및 격리 검증을 위한 20가지 Docker Compose 샌드박스 구성을 체계적으로 정리한 실전 자료임. 각 구성은 특정 아키텍처나 검증 목적을 갖도록 설계하며, 서비스 간 통신은 Docker 내부 네트워크를 기본으로 하고 필요한 포트만 호스트에 공개하는 방식을 사용함.

이 문서에서 고친 곳

- root 비밀번호만 정하고 PermitRootLogin 을 바꾸지 않던 예제 4곳(LLM·TUI·크로스 컴파일·관리 콘솔)에 sed 한 줄을 더했습니다. 기본값(prohibit-password)에서는 root 비밀번호 SSH 로그인이 거부됩니다. 다른 예제들이 이미 쓰던 줄과 같습니다.
- 문서 생성 과정의 대화 흔적(「사용자가 제공한」)을 지웠습니다.

1. 문서 사용 전 중요 보안 및 정확성 안내

- 아래 예제의 비밀번호는 학습용 예시임. 실제 환경에서는 .env, Docker secrets 또는 별도 Secret Manager를 사용함.
- SSH에서 root 비밀번호 로그인을 허용하는 예제는 격리된 로컬 개발 검증용으로만 사용함. 장기 사용 시 일반 사용자 + SSH public key 방식으로 변경하는 것이 바람직함.
- OpenAI API 키를 docker-compose.yml에 직접 적지 않고 `${OPENAI_API_KEY}`처럼 환경변수로 주입함.
- Codex CLI 설치 명령은 설치 시점의 공식 패키지명과 CLI 버전에 따라 달라질 수 있으므로 실제 구축 시 공식 OpenAI 문서의 현재 설치 명령을 확인함.
- MariaDB master/slave 예제는 컨테이너 두 개를 배치한 골격임. 실제 복제 검증에는 server-id, binlog, replication user, CHANGE MASTER/CHANGE REPLICATION SOURCE 등의 추가 초기화가 필요함.
- Docker Compose 최신 구현에서는 최상위 `version:` 필드가 필수가 아님. 아래 예제에도 version: 필드는 넣지 않았음.

2. 20가지 구성 한눈에 보기

No.	구성	핵심 조합
1	올인원 통합 개발	Codex/SSH + Kotlin + Rust + MariaDB
2	보안 원격 Rust 개발	SSH + Rust + Codex
3	DB 복제 검증	Spring Boot + MariaDB Master/Replica
4	고성능 비동기	Rust Axum/Tokio + MariaDB + Redis
5	로컬 LLM	Ollama + Codex + SSH
6	Polyglot RPC	Spring WebFlux + Rust gRPC + MariaDB
7	CI/CD 테스트	Kotlin Gradle + Rust Cargo + 임시 DB
8	Bastion 격리	SSH Bastion + 내부 DB/API
9	시계열/연산	Rust + MariaDB + Redis Pub/Sub

No.	구성	핵심 조합
10	TUI 관리	Rust Ratatui + Spring Admin
11	포렌식	Rust parser + Kotlin reporter + MariaDB
12	이벤트 메시징	Redpanda/Kafka + Rust + Kotlin
13	ARM 크로스빌드	aarch64 Rust + SSH
14	API Gateway	Traefik + Kotlin + Rust
15	Hardened	Non-root + cap_drop + resource limits
16	멀티테넌트	2 MariaDB + Kotlin routing
17	Job Queue	Redis + Kotlin producer + Rust worker
18	WebSocket	Rust WebSocket + Kotlin simulator
19	Cloud IDE	Code-Server + Codex + Rust + JDK
20	프로덕션 모사	MariaDB + Redis + SSH/Codex + Spring + Rust + Prometheus

섹션 1. 올인원 통합 개발 샌드박스

SSH 터미널, Codex CLI, Kotlin/Spring Boot, Rust 워커, MariaDB가 하나의 bridge 네트워크에서 연결되는 표준 풀스택 개발 환경임. MariaDB는 named volume으로 영구 저장하고 Spring Boot와 Rust는 서비스명 `mariadb`로 DB에 접근함.

핵심 포인트: 표준 학습용 풀스택 · DB 데이터 영구 보존 · Codex 작업 폴더 host mount · 8080/2222/3306 외부 접속

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    container_name: sbx-mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: app_sandbox
      MYSQL_USER: devuser
      MYSQL_PASSWORD: ${DB_APP_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
    networks: [sandbox-net]
    healthcheck:
      test: ["CMD", "mariadb-admin", "ping", "-h", "localhost", "-u", "root", "-p${DB_ROOT_PAS
SWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10

  ssh-codex-dev:
    image: ubuntu:24.04
    container_name: sbx-ssh-codex
    restart: unless-stopped
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server curl git build-essential
      pkg-config libssl-dev nodejs npm &&
      mkdir -p /var/run/sshd &&
      echo 'root:${SSH_DEV_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
      /etc/ssh/sshd_config &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    ports:
      - "2222:22"
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    volumes:
      - ./workspace:/workspace
    working_dir: /workspace
    networks: [sandbox-net]

  kotlin-springboot:
    image: eclipse-temurin:21-jdk-jammy
    container_name: sbx-springboot
    working_dir: /app
    volumes:
      - ./backend-kotlin:/app
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/app_sandbox
      SPRING_DATASOURCE_USERNAME: devuser
      SPRING_DATASOURCE_PASSWORD: ${DB_APP_PASSWORD}
    command: sh -c "./gradlew bootRun"
```

```
    ports:
      - "8080:8080"
    depends_on:
      mariadb:
        condition: service_healthy
    networks: [sandbox-net]

  rust-worker:
    image: rust:1.80-bullseye
    container_name: sbx-rust-worker
    working_dir: /app
    volumes:
      - ./worker-rust:/app
    environment:
      DATABASE_URL: mysql://devuser:${DB_APP_PASSWORD}@mariadb:3306/app_sandbox
    command: bash -c "cargo install cargo-watch && cargo watch -x run"
    depends_on:
      mariadb:
        condition: service_healthy
    networks: [sandbox-net]

volumes:
  mariadb_data:

networks:
  sandbox-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 2. 보안 원격 개발 컨테이너 (SSH + Rust + Codex)

Rust 공식 이미지를 기반으로 SSH와 Codex를 추가하여 원격 터미널 중심으로 개발하는 구성임. Cargo cache를 named volume으로 유지하여 반복 빌드 시간을 줄이는 목적임.

핵심 포인트: Rust 원격 개발 · Cargo cache 영구화 · SSH 2201 · Codex CLI

Docker Compose 예제

```yaml
services:
  remote-codex-rust:
    image: rust:1.80-slim-bookworm
    container_name: sbx-remote-rust-cli
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    ports:
      - "2201:22"
    volumes:
      - ./rust_project:/root/project
      - rust_cargo_cache:/usr/local/cargo
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server nodejs npm git zsh curl &&
      echo 'root:${SSH_RUST_PASSWORD}' | chpasswd &&
      mkdir -p /var/run/sshd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
      /etc/ssh/sshd_config &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    networks: [isolated-dev]

volumes:
  rust_cargo_cache:

networks:
  isolated-dev:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 3. Kotlin/Spring Boot & MariaDB 레플리카 테스트 환경

Spring Boot에서 쓰기 DB와 읽기 DB를 분리하는 애플리케이션 구조를 검증하기 위한 골격임. 단, 컨테이너 두 개를 띄우는 것만으로 MariaDB 복제가 자동 구성되지는 않으므로 실제 replication 설정 파일과 초기화 SQL이 추가로 필요함.

핵심 포인트: 다중 datasource · 읽기/쓰기 라우팅 검증 · 실제 replication 초기화는 별도

Docker Compose 예제

```yaml
services:
  mariadb-master:
    image: mariadb:11.2
    container_name: sbx-mariadb-master
    environment:
      MYSQL_ROOT_PASSWORD: ${MASTER_ROOT_PASSWORD}
      MYSQL_DATABASE: replication_db
      MYSQL_USER: repl_user
      MYSQL_PASSWORD: ${REPL_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - db_master_data:/var/lib/mysql
    networks: [repl-net]

  mariadb-slave:
    image: mariadb:11.2
    container_name: sbx-mariadb-slave
    environment:
      MYSQL_ROOT_PASSWORD: ${SLAVE_ROOT_PASSWORD}
    ports:
      - "3307:3306"
    volumes:
      - db_slave_data:/var/lib/mysql
    networks: [repl-net]

  kotlin-app:
    image: gradle:8.7-jdk21
    container_name: sbx-kotlin-multi-ds
    working_dir: /app
    volumes:
      - ./app-src:/app
    environment:
      SPRING_DATASOURCE_WRITE_URL: jdbc:mariadb://mariadb-master:3306/replication_db
      SPRING_DATASOURCE_READ_URL: jdbc:mariadb://mariadb-slave:3306/replication_db
      DB_USER: repl_user
      DB_PASSWORD: ${REPL_PASSWORD}
    command: gradle bootRun
    ports:
      - "8080:8080"
    networks: [repl-net]

volumes:
  db_master_data:
  db_slave_data:

networks:
  repl-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 4. 고성능 Rust 비동기 백엔드 & MariaDB 풀링

Axum/Tokio 서비스에 MariaDB와 Redis를 결합하여 DB connection pool, 비동기 처리, cache hit/miss를 실험하는 구성임.

핵심 포인트: Axum/Tokio · Redis cache · MariaDB · Rust release 실행

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:10.11
    container_name: sbx-perf-db
    environment:
      MYSQL_ROOT_PASSWORD: ${PERF_DB_PASSWORD}
      MYSQL_DATABASE: async_perf
    ports:
      - "3308:3306"
    volumes:
      - perf_data:/var/lib/mysql
    networks: [perf-net]

  redis:
    image: redis:7-alpine
    container_name: sbx-perf-redis
    ports:
      - "6379:6379"
    networks: [perf-net]

  rust-axum:
    image: rust:1.80-bullseye
    container_name: sbx-rust-axum
    working_dir: /service
    volumes:
      - ./rust_axum_service:/service
    environment:
      DATABASE_URL: mysql://root:${PERF_DB_PASSWORD}@mariadb:3306/async_perf
      REDIS_URL: redis://redis:6379
    command: bash -c "cargo run --release"
    ports:
      - "9000:9000"
    depends_on: [mariadb, redis]
    networks: [perf-net]

volumes:
  perf_data:

networks:
  perf-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 5. 로컬 LLM(Ollama) + Codex CLI + SSH 협업 환경

Ollama 모델 저장소를 named volume으로 유지하고 SSH 개발 노드와 같은 내부 네트워크에 배치하는 실험형 구성임.
Codex 자체의 모델 공급자/endpoint 호환성은 사용하는 Codex 버전과 설정을 별도로 확인해야 함.

핵심 포인트: Ollama 11434 · Codex/SSH 개발노드 · 모델 volume 영구화

Docker Compose 예제

```yaml
services:
  ollama-engine:
    image: ollama/ollama:latest
    container_name: sbx-ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_models:/root/.ollama
    networks: [llm-net]

  ssh-codex:
    image: debian:12-slim
    container_name: sbx-ssh-llm-cli
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      OLLAMA_URL: http://ollama-engine:11434
    ports:
      - "2223:22"
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server git curl nodejs npm &&
      echo 'root:${SSH_LLM_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    depends_on: [ollama-engine]
    networks: [llm-net]

volumes:
  ollama_models:

networks:
  llm-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 6. Polyglot Microservices: Spring Boot Gateway + Rust gRPC

Spring WebFlux 계층과 Rust gRPC 엔진을 분리하여 언어 간 RPC 및 장애 전파를 검증함.

핵심 포인트: gRPC · WebFlux · Polyglot

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${POLYGLOT_DB_PASSWORD}
      MYSQL_DATABASE: polyglot_db
    networks: [internal-rpc]

  rust-grpc-core:
    image: rust:1.80-bullseye
    working_dir: /app
    volumes: [./rust_grpc:/app]
    environment:
      DATABASE_URL: mysql://root:${POLYGLOT_DB_PASSWORD}@mariadb:3306/polyglot_db
    command: cargo run --bin server
    ports: ["50051:50051"]
    depends_on: [mariadb]
    networks: [internal-rpc]

  kotlin-gateway:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /gateway
    volumes: [./gateway-src:/gateway]
    environment:
      GRPC_SERVER_HOST: rust-grpc-core
      GRPC_SERVER_PORT: "50051"
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    depends_on: [rust-grpc-core]
    networks: [internal-rpc]

networks:
  internal-rpc:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 7. 자동화 CI/CD & 테스트 러너

Kotlin Gradle check와 Rust cargo test를 격리 컨테이너에서 수행하고 DB는 tmpfs로 휘발시킴.

핵심 포인트: CI 시뮬레이션 · tmpfs DB · Kotlin/Rust 병렬 검증

Docker Compose 예제

```yaml
services:
  ephemeral-mariadb:
    image: mariadb:10.11
    environment:
      MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
      MYSQL_DATABASE: ci_test
    tmpfs: [/var/lib/mysql]
    networks: [ci-net]

  test-runner-kotlin:
    image: gradle:8.7-jdk21
    working_dir: /ci
    volumes: [./kotlin-project:/ci]
    environment:
      TEST_DB_HOST: ephemeral-mariadb
    command: gradle check --info
    depends_on: [ephemeral-mariadb]
    networks: [ci-net]

  test-runner-rust:
    image: rust:1.80
    working_dir: /ci
    volumes: [./rust-project:/ci]
    environment:
      DATABASE_URL: mysql://root@ephemeral-mariadb:3306/ci_test
    command: cargo test --verbose
    depends_on: [ephemeral-mariadb]
    networks: [ci-net]

networks:
  ci-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

섹션 8. Bastion Host 기반 보안 서브넷

외부에서는 Bastion만 접근하고 DB와 API는 internal network에 두어 직접 노출을 막는 구성임.

핵심 포인트: Bastion · internal network · 최소 포트 공개

Docker Compose 예제

```yaml
services:
  ssh-bastion:
    image: linuxserver/openssh-server:latest
    environment:
      PUID: "1000"
      PGID: "1000"
      PASSWORD_ACCESS: "true"
      USER_PASSWORD: ${BASTION_PASSWORD}
      USER_NAME: developer
    ports: ["2224:2222"]
    networks: [dmz-net, secure-backend]

  mariadb-internal:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${INTERNAL_DB_PASSWORD}
      MYSQL_DATABASE: internal_core
    networks: [secure-backend]

  kotlin-backend:
    image: eclipse-temurin:21-jre-alpine
    environment:
      DB_HOST: mariadb-internal
    networks: [secure-backend]

networks:
  dmz-net:
    driver: bridge
  secure-backend:
    internal: true
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 9. Rust 고빈도 연산 & 시계열 저장

Rust 수집/연산 엔진과 Redis Pub/Sub, MariaDB 저장 계층을 조합함.

핵심 포인트: 고빈도 연산 · Redis Pub/Sub · MariaDB tuning

Docker Compose 예제

```yaml
services:
  mariadb-timeseries:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TS_DB_PASSWORD}
      MYSQL_DATABASE: metrics_ts
    ports: ["3309:3306"]
    command: --innodb-buffer-pool-size=512M --innodb-flush-log-at-trx-commit=2
    networks: [quant-net]

  redis-feed:
    image: redis:7-alpine
    ports: ["6380:6379"]
    networks: [quant-net]

  rust-engine:
    image: rust:1.80-bullseye
    working_dir: /quant
    volumes: [./rust_quant:/quant]
    environment:
      DB_URL: mysql://root:${TS_DB_PASSWORD}@mariadb-timeseries:3306/metrics_ts
      REDIS_URL: redis://redis-feed:6379
    command: cargo run --release --bin market_collector
    networks: [quant-net]

networks:
  quant-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 10. SSH TUI 관리자 & Spring Boot Admin

Rust Ratatui CLI 관리 도구와 Spring 관리 API를 같은 DB에 연결하는 운영도구 실험 환경임.

핵심 포인트: Ratatui · Spring Admin · SSH

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TUI_DB_PASSWORD}
      MYSQL_DATABASE: manage_center
    networks: [tui-net]

  admin-spring:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /srv
    volumes: [./spring-admin:/srv]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/manage_center
    command: ./gradlew bootRun
    ports: ["8081:8080"]
    networks: [tui-net]

  ssh-rust-tui:
    image: rust:1.80-bullseye
    ports: ["2225:22"]
    volumes: [./rust-tui:/opt/tui]
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server cmake pkg-config libncurses5-dev &&
      echo 'root:${TUI_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      /usr/sbin/sshd -D"
    networks: [tui-net]

networks:
  tui-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 11. 데이터 포렌식 및 바이너리 파싱

증거 샘플은 read-only로 mount하고 Rust parser가 분석 메타데이터를 MariaDB에 저장하며 Kotlin reporter가 결과를 생성함.

핵심 포인트: read-only evidence · Rust parser · Kotlin reporter

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${FORENSICS_DB_PASSWORD}
      MYSQL_DATABASE: forensics_evidence
    volumes: [forensic_db_vol:/var/lib/mysql]
    networks: [parse-net]

  rust-parser:
    image: rust:1.80-bullseye
    working_dir: /analysis
    volumes:
      - ./evidence_samples:/samples:ro
      - ./rust_parser:/analysis
    environment:
      DB_URL: mysql://root:${FORENSICS_DB_PASSWORD}@mariadb:3306/forensics_evidence
    command: cargo run --bin raw_disk_parser
    networks: [parse-net]

  kotlin-reporter:
    image: eclipse-temurin:21-jre
    working_dir: /reporter
    volumes: [./reporter_build:/reporter]
    environment:
      DB_HOST: mariadb
    command: java -jar reporter.jar
    networks: [parse-net]

volumes:
  forensic_db_vol:

networks:
  parse-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 12. 이벤트 기반 메시징 (Redpanda/Kafka)

Kafka 호환 Redpanda를 중심으로 Rust producer와 Kotlin consumer를 연결하는 이벤트 기반 검증 환경임.

핵심 포인트: Kafka 호환 · producer/consumer · event store

Docker Compose 예제

```yaml
services:
  redpanda:
    image: docker.redpanda.com/redpandadata/redpanda:v24.1.1
    command:
      - redpanda
      - start
      - --smp
      - "1"
      - --memory
      - 512M
      - --overprovisioned
      - --kafka-addr
      - internal://0.0.0.0:9092,external://0.0.0.0:19092
      - --advertise-kafka-addr
      - internal://redpanda:9092,external://localhost:19092
    ports: ["19092:19092"]
    networks: [event-net]

  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${EVENT_DB_PASSWORD}
      MYSQL_DATABASE: event_store
    networks: [event-net]

  rust-producer:
    image: rust:1.80-slim
    working_dir: /producer
    volumes: [./rust_producer:/producer]
    environment:
      KAFKA_BROKERS: redpanda:9092
    command: cargo run
    depends_on: [redpanda]
    networks: [event-net]

  kotlin-consumer:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /consumer
    volumes: [./kotlin_consumer:/consumer]
    environment:
      KAFKA_SERVER: redpanda:9092
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/event_store
    command: ./gradlew bootRun
    depends_on: [redpanda, mariadb]
    networks: [event-net]

networks:
  event-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 13. ARM/Embedded 크로스 컴파일

x86_64 호스트에서 aarch64 타깃 Rust 빌드를 검증하는 크로스 컴파일 환경임.

핵심 포인트: aarch64 · Jetson/Embedded · SSH build node

Docker Compose 예제

```yaml
services:
  cross-build-server:
    image: ghcr.io/cross-rs/aarch64-unknown-linux-gnu:edge
    environment:
      TARGET: aarch64-unknown-linux-gnu
    ports: ["2226:22"]
    volumes: [./embedded_rust:/workspace]
    working_dir: /workspace
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server &&
      echo 'root:${CROSS_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      /usr/sbin/sshd -D"
    networks: [cross-net]

networks:
  cross-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 14. Traefik API Gateway

Traefik이 Docker labels를 읽어 `/api/v1/user`는 Kotlin, `/api/v1/compute`는 Rust로 라우팅함.

핵심 포인트: Reverse proxy · path routing · Docker socket read-only

Docker Compose 예제

```yaml
services:
  traefik:
    image: traefik:v3.0
    command:
      - --api.insecure=true
      - --providers.docker=true
      - --entrypoints.web.address=:80
    ports:
      - "80:80"
      - "8082:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
    networks: [edge-net]

  kotlin-service:
    image: eclipse-temurin:21-jre-alpine
    volumes: [./kotlin_jar:/app]
    command: java -jar /app/app.jar
    labels:
      - traefik.enable=true
      - traefik.http.routers.kotlin.rule=PathPrefix(`/api/v1/user`)
      - traefik.http.services.kotlin.loadbalancer.server.port=8080
    networks: [edge-net]

  rust-service:
    image: rust:1.80-slim
    volumes: [./rust_bin:/app]
    command: /app/service_binary
    labels:
      - traefik.enable=true
      - traefik.http.routers.rust.rule=PathPrefix(`/api/v1/compute`)
      - traefik.http.services.rust.loadbalancer.server.port=9000
    networks: [edge-net]

networks:
  edge-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

섹션 15. Hardened 루트리스 개발

Non-root, cap_drop, no-new-privileges, resource limit을 사용해 보안 제약 하에서 Rust 테스트를 수행함.

핵심 포인트: Non-root · cap_drop ALL · resource limits

Docker Compose 예제

```yaml
services:
  hardened-mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${HARDENED_DB_PASSWORD}
      MYSQL_DATABASE: secure_zone
    mem_limit: 1024m
    cpus: 1.5
    networks: [hardened-net]

  hardened-rust:
    image: rust:1.80-slim
    user: "1001:1001"
    working_dir: /home/appuser/src
    volumes: [./secure_code:/home/appuser/src]
    command: cargo test
    cap_drop: [ALL]
    security_opt:
      - no-new-privileges:true
    mem_limit: 512m
    cpus: 1.0
    networks: [hardened-net]

networks:
  hardened-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 16. 멀티 테넌트 격리 DB

Tenant A/B의 DB를 별도 컨테이너로 분리하고 Kotlin에서 tenant별 datasource routing을 검증함.

핵심 포인트: Tenant isolation · dynamic datasource · 2 DB

Docker Compose 예제

```yaml
services:
  mariadb-tenant-a:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TENANT_A_PASSWORD}
      MYSQL_DATABASE: tenant_a_db
    networks: [tenant-net]

  mariadb-tenant-b:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TENANT_B_PASSWORD}
      MYSQL_DATABASE: tenant_b_db
    networks: [tenant-net]

  kotlin-router:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /app
    volumes: [./multitenant-src:/app]
    environment:
      TENANT_A_URL: jdbc:mariadb://mariadb-tenant-a:3306/tenant_a_db
      TENANT_B_URL: jdbc:mariadb://mariadb-tenant-b:3306/tenant_b_db
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    networks: [tenant-net]

networks:
  tenant-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 17. 분산 작업 큐 & 배치 워커

Kotlin producer가 Redis에 작업을 발행하고 Rust consumer가 처리 후 MariaDB에 저장하는 비동기 배치 구조임.

핵심 포인트: Redis queue · Kotlin producer · Rust worker

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${BATCH_DB_PASSWORD}
      MYSQL_DATABASE: batch_store
    networks: [job-net]

  redis-queue:
    image: redis:7-alpine
    networks: [job-net]

  kotlin-job-producer:
    image: eclipse-temurin:21-jre-alpine
    volumes: [./producer_jar:/app]
    environment:
      REDIS_HOST: redis-queue
    command: java -jar /app/producer.jar
    networks: [job-net]

  rust-job-consumer:
    image: rust:1.80-bullseye
    working_dir: /worker
    volumes: [./rust_worker:/worker]
    environment:
      REDIS_URL: redis://redis-queue:6379
      DATABASE_URL: mysql://root:${BATCH_DB_PASSWORD}@mariadb:3306/batch_store
    command: cargo run --bin queue_worker
    networks: [job-net]

networks:
  job-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 18. 고성능 실시간 WebSocket

Rust WebSocket 서버와 Kotlin client simulator를 연결하고 세션 메타데이터를 MariaDB에 저장함.

핵심 포인트: WebSocket · Tokio · client simulator

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${WS_DB_PASSWORD}
      MYSQL_DATABASE: ws_sessions
    networks: [ws-net]

  rust-ws-server:
    image: rust:1.80-bullseye
    working_dir: /ws
    volumes: [./rust_ws:/ws]
    environment:
      DATABASE_URL: mysql://root:${WS_DB_PASSWORD}@mariadb:3306/ws_sessions
    command: cargo run --release
    ports: ["9001:9001"]
    networks: [ws-net]

  kotlin-client-sim:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /sim
    volumes: [./client_simulator:/sim]
    environment:
      WS_TARGET: ws://rust-ws-server:9001
    command: ./gradlew run
    depends_on: [rust-ws-server]
    networks: [ws-net]

networks:
  ws-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 19. 클라우드 IDE / Code-Server + Codex

브라우저 IDE, Codex CLI, JDK, Rust를 하나의 개발 컨테이너에 넣고 MariaDB를 별도 서비스로 연결하는 구성임.

핵심 포인트: Browser IDE · Codex · Rust/JDK · MariaDB

Docker Compose 예제

```yaml
services:
  code-server-sandbox:
    image: codercom/code-server:latest
    ports: ["8443:8080"]
    environment:
      PASSWORD: ${IDE_PASSWORD}
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    volumes:
      - ./ide_workspace:/home/coder/project
      - ide_config:/home/coder/.config
    entrypoint: >
      bash -c "sudo apt-get update &&
      sudo apt-get install -y default-jdk build-essential curl nodejs npm &&
      curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs |
      sh -s -- -y &&
      sudo npm install -g @openai/codex &&
      /usr/bin/entrypoint.sh --bind-addr 0.0.0.0:8080
      /home/coder/project"
    networks: [ide-net]

  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${IDE_DB_PASSWORD}
      MYSQL_DATABASE: dev_db
    ports: ["3310:3306"]
    networks: [ide-net]

volumes:
  ide_config:

networks:
  ide-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 20. 로컬 프로덕션 모사 풀스택

MariaDB, Redis, 관리 SSH/Codex, Spring Boot, Rust worker, Prometheus를 통합하여 운영과 유사한 의존관계를 로컬에서 검증함.

핵심 포인트: 운영 모사 · Prometheus · Redis · Spring/Rust · 관리 SSH/Codex

Docker Compose 예제

```yaml
services:
  mariadb:
    image: mariadb:11.2
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${PROD_DB_ROOT_PASSWORD}
      MYSQL_DATABASE: prod_cluster
      MYSQL_USER: service_app
      MYSQL_PASSWORD: ${PROD_DB_APP_PASSWORD}
    volumes: [prod_db_data:/var/lib/mysql]
    ports: ["3306:3306"]
    networks: [full-cluster]

  redis:
    image: redis:7.2-alpine
    networks: [full-cluster]

  ssh-management-node:
    image: ubuntu:24.04
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server git curl mariadb-client nodejs npm &&
      mkdir -p /var/run/sshd &&
      echo 'root:${MGMT_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    ports: ["2227:22"]
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    networks: [full-cluster]

  springboot-core:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /workspace
    volumes: [./springboot_core:/workspace]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/prod_cluster
      SPRING_DATASOURCE_USERNAME: service_app
      SPRING_DATASOURCE_PASSWORD: ${PROD_DB_APP_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_DATA_REDIS_PORT: "6379"
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    depends_on: [mariadb, redis]
    networks: [full-cluster]

  rust-analyzer-worker:
    image: rust:1.80-bullseye
    working_dir: /analyzer
    volumes: [./rust_analyzer:/analyzer]
    environment:
      DATABASE_URL: mysql://service_app:${PROD_DB_APP_PASSWORD}@mariadb:3306/prod_cluster
      REDIS_URL: redis://redis:6379
    command: cargo run --release
```

```
    depends_on: [mariadb, redis]
    networks: [full-cluster]

  prometheus:
    image: prom/prometheus:v2.51.0
    ports: ["9090:9090"]
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    networks: [full-cluster]

volumes:
  prod_db_data:

networks:
  full-cluster:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```bash
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

21. 공통 .env 파일 작성

문서에 포함된 `${...}` 값은 실제 비밀번호나 API 키를 Compose 파일에 직접 기록하지 않도록 분리한 것임. 아래는 예시이며 실제 값으로 교체하고 `.env`는 Git에 커밋하지 않음.

```env
OPENAI_API_KEY=YOUR_OPENAI_API_KEY

DB_ROOT_PASSWORD=CHANGE_ME
DB_APP_PASSWORD=CHANGE_ME
SSH_DEV_PASSWORD=CHANGE_ME
SSH_RUST_PASSWORD=CHANGE_ME
MASTER_ROOT_PASSWORD=CHANGE_ME
SLAVE_ROOT_PASSWORD=CHANGE_ME
REPL_PASSWORD=CHANGE_ME
PERF_DB_PASSWORD=CHANGE_ME
SSH_LLM_PASSWORD=CHANGE_ME
POLYGLOT_DB_PASSWORD=CHANGE_ME
BASTION_PASSWORD=CHANGE_ME
INTERNAL_DB_PASSWORD=CHANGE_ME
TS_DB_PASSWORD=CHANGE_ME
TUI_DB_PASSWORD=CHANGE_ME
TUI_SSH_PASSWORD=CHANGE_ME
FORENSICS_DB_PASSWORD=CHANGE_ME
EVENT_DB_PASSWORD=CHANGE_ME
CROSS_SSH_PASSWORD=CHANGE_ME
HARDENED_DB_PASSWORD=CHANGE_ME
TENANT_A_PASSWORD=CHANGE_ME
TENANT_B_PASSWORD=CHANGE_ME
BATCH_DB_PASSWORD=CHANGE_ME
WS_DB_PASSWORD=CHANGE_ME
IDE_PASSWORD=CHANGE_ME
IDE_DB_PASSWORD=CHANGE_ME
PROD_DB_ROOT_PASSWORD=CHANGE_ME
PROD_DB_APP_PASSWORD=CHANGE_ME
MGMT_SSH_PASSWORD=CHANGE_ME
```

22. 공통 서버 구동 및 검증 절차

22.1 Compose 문법 검사

```bash
docker compose config
```

YAML 들여쓰기, 변수 치환, Compose schema 문제를 먼저 확인함.

22.2 백그라운드 구동

```bash
docker compose up -d --build
docker compose ps
```

22.3 로그 확인

```bash
docker compose logs -f
docker compose logs --tail=200 mariadb
docker compose logs --tail=200 kotlin-springboot
```

22.4 SSH 접속

```bash
ssh root@localhost -p 2222
```

실제 포트는 각 섹션의 Compose 설정을 확인함. 로컬 실습이 아니라면 root password SSH 대신 key 인증과 일반 사용자를 권장함.

22.5 Codex CLI 확인

```bash
docker exec -it sbx-ssh-codex codex --version
docker exec -it sbx-ssh-codex codex
```

Codex CLI의 설치/인증 방식은 버전에 따라 변경될 수 있으므로 실제 구축 시 현재 공식 OpenAI Codex 문서를 확인함.

22.6 MariaDB 확인

```bash
docker exec -it sbx-mariadb   mariadb -u devuser -p -e "SHOW DATABASES;"

docker compose exec mariadb   mariadb-admin ping -h localhost
```

22.7 Spring Boot 확인

```bash
curl -v http://localhost:8080/
docker compose logs -f kotlin-springboot
```

22.8 Rust 확인

```bash
docker compose exec rust-worker rustc --version
docker compose exec rust-worker cargo --version
docker compose logs -f rust-worker
```

22.9 네트워크 확인

```bash
docker network ls
docker compose exec kotlin-springboot getent hosts mariadb
docker compose exec rust-worker getent hosts mariadb
```

22.10 종료

```bash
# 컨테이너/네트워크 제거, named volume 유지
docker compose down

# named volume까지 제거 - DB 데이터 삭제 주의
docker compose down -v
```

23. 개발/격리 검증 체크리스트

- 외부에 꼭 필요한 포트만 publish했는가?
- DB 접속 시 컨테이너 간에는 서비스명:내부포트를 사용하는가?
- 비밀번호/API 키가 Compose 파일과 Git에 하드코딩되어 있지 않은가?
- DB가 영구 보존되어야 하면 named volume이 있는가?
- CI용 DB는 tmpfs 등 휘발성 저장이 더 적합한가?
- 증거/원본 파일은 read-only mount가 필요한가?
- Bastion/secure backend에는 internal network가 적절한가?
- Rust/Kotlin 빌드 cache를 volume으로 분리할 필요가 있는가?
- healthcheck가 필요한 핵심 의존 서비스에 설정되어 있는가?

- 실제 운영 전에는 latest 태그 대신 검증된 고정 버전을 사용하는가?

24. 권장 사용 구분

목적	권장 섹션
일반 Kotlin/Rust 풀스택 개발	1
SSH 원격 Rust 개발	2
Spring 다중 DB/복제 테스트	3
Rust 성능/Redis	4, 9
로컬 LLM	5
Kotlin-Rust RPC	6
CI 테스트	7
네트워크 격리	8, 15
포렌식/파일 분석	11
Kafka 이벤트 처리	12
Jetson/aarch64 빌드	13
Gateway/Reverse Proxy	14
멀티테넌트	16
비동기 Job Queue	17
WebSocket	18
웹 IDE	19
통합 운영 모사	20

부록 · Docker 샌드박스 · Compose

Docker Sandbox 명령어 100선

Docker Compose + SSH + Linux + Spring Boot Kotlin + MariaDB + Codex AI 초보자 실전 교본

사용 기준: 현재 구성한 sbx-kotlin / sbx-mariadb 환경을 기준으로 작성함. Docker Sandbox 자체의 세부 하위 명령은 Docker 버전에 따라 달라질 수 있으므로 `docker sandbox --help`에서 현재 설치본의 명령을 먼저 확인함.

1. 명령어 100개 표

No.	명령어	설명
1	docker --version	Docker 버전 확인
2	docker version	Docker Client/Server 상세 버전 확인
3	docker info	Docker 실행환경 전체 정보 확인
4	docker sandbox --help	현재 설치된 Docker Sandbox 지원 명령 확인
5	docker compose version	Docker Compose 버전 확인
6	docker ps	실행 중인 컨테이너 확인
7	docker ps -a	정지된 것을 포함한 전체 컨테이너 확인
8	docker images	로컬 Docker 이미지 목록 확인
9	docker network ls	Docker 네트워크 목록 확인
10	docker volume ls	Docker 볼륨 목록 확인
11	docker compose up	compose.yml 서비스를 포그라운드로 실행
12	docker compose up -d	compose.yml 서비스를 백그라운드 실행
13	docker compose up --build -d	이미지를 다시 빌드한 후 백그라운드 실행
14	docker compose build	compose.yml의 이미지 빌드
15	docker compose build --no-cache	캐시 없이 이미지를 처음부터 다시 빌드
16	docker compose ps	Compose 서비스 상태 확인
17	docker compose stop	Compose 서비스 정지
18	docker compose start	정지한 Compose 서비스 시작
19	docker compose restart	Compose 서비스 재시작
20	docker compose down	Compose 컨테이너와 네트워크 종료/삭제
21	docker compose down -v	Compose 종료 및 볼륨까지 삭제 - DB 데이터 삭제 주의
22	docker compose config	compose.yml 문법과 최종 적용 설정 확인
23	docker compose logs	Compose 전체 로그 확인
24	docker compose logs -f	Compose 전체 로그 실시간 확인
25	docker compose logs -f app	app 서비스 로그 실시간 확인
26	docker compose logs -f mariadb	MariaDB 서비스 로그 실시간 확인

No.	명령어	설명
27	docker compose exec app bash	실행 중인 app 컨테이너에 Bash로 접속
28	docker compose exec mariadb bash	실행 중인 MariaDB 컨테이너에 Bash로 접속
29	docker compose top	Compose 서비스 내부 프로세스 확인
30	docker compose pull	서비스 이미지 최신 버전 내려받기
31	docker exec -it sbx-kotlin bash	sbx-kotlin 컨테이너에 직접 Bash 접속
32	docker exec -it sbx-mariadb bash	sbx-mariadb 컨테이너에 직접 Bash 접속
33	docker inspect sbx-kotlin	Kotlin 컨테이너 상세 설정 확인
34	docker inspect sbx-mariadb	MariaDB 컨테이너 상세 설정 확인
35	docker stats	모든 실행 컨테이너 CPU/RAM 실시간 확인
36	docker stats sbx-kotlin	Kotlin 컨테이너 자원 사용량 확인
37	docker stats sbx-mariadb	MariaDB 컨테이너 자원 사용량 확인
38	docker logs sbx-kotlin	Kotlin 컨테이너 로그 확인
39	docker logs -f sbx-kotlin	Kotlin 컨테이너 로그 실시간 확인
40	docker logs -f sbx-mariadb	MariaDB 컨테이너 로그 실시간 확인
41	docker start sbx-kotlin	sbx-kotlin 컨테이너 시작
42	docker stop sbx-kotlin	sbx-kotlin 컨테이너 정지
43	docker restart sbx-kotlin	sbx-kotlin 컨테이너 재시작
44	docker rm sbx-kotlin	정지된 sbx-kotlin 컨테이너 삭제
45	docker rm -f sbx-kotlin	sbx-kotlin 컨테이너 강제 삭제
46	docker image prune	사용하지 않는 이미지 정리
47	docker container prune	정지된 컨테이너 일괄 정리
48	docker volume prune	사용하지 않는 볼륨 정리 - 데이터 삭제 주의
49	docker network prune	사용하지 않는 네트워크 정리
50	docker system df	Docker가 사용하는 디스크 공간 확인
51	docker system prune	미사용 Docker 리소스 정리
52	ssh -p 2222 devuser@127.0.0.1	같은 PC에서 SSH로 개발 컨테이너 접속
53	ssh -p 2222 devuser@SERVER_IP	다른 LAN PC에서 Docker 호스트로 SSH 접속 예
54	whoami	SSH 접속 후 현재 사용자 확인
55	hostname	컨테이너 호스트명 확인
56	pwd	현재 작업 디렉터리 확인
57	ls	현재 디렉터리 파일 목록
58	ls -al	숨김파일까지 포함하여 상세 목록 확인
59	cd /workspace	Spring Boot 프로젝트 작업폴더로 이동

No.	명령어	설명
60	df -h	컨테이너 디스크 사용량 확인
61	free -mh	메모리 사용량 확인
62	ps aux	실행 중인 프로세스 전체 확인
63	top	CPU/메모리/프로세스 실시간 확인
64	ip addr	컨테이너 IP와 네트워크 인터페이스 확인
65	ss -lntp	TCP LISTEN 포트와 프로세스 확인
66	ss -lntp | grep 8080	Spring Boot 8080 포트 LISTEN 확인
67	ss -lntp | grep ':22'	SSH 22 포트 LISTEN 확인
68	ping mariadb	app 컨테이너에서 MariaDB 서비스명 통신 확인
69	curl http://127.0.0.1:8080	컨테이너 내부에서 Spring Boot REST API 테스트
70	git --version	Git 설치 및 버전 확인
71	git status	현재 Git 변경사항 확인
72	git diff	수정된 코드 차이 확인
73	git log --oneline	Git 커밋 이력을 한 줄씩 확인
74	java -version	JDK 실행 버전 확인
75	javac -version	Java 컴파일러 버전 확인
76	./gradlew --version	프로젝트 Gradle Wrapper 버전 확인
77	chmod +x gradlew	Linux에서 Gradle Wrapper 실행권한 부여
78	./gradlew clean	기존 build 결과 삭제
79	./gradlew build	컴파일·테스트·패키징 전체 빌드
80	./gradlew clean build	기존 결과 삭제 후 전체 빌드
81	./gradlew test	테스트 코드 실행
82	./gradlew bootRun	Spring Boot 애플리케이션 실행
83	./gradlew tasks	사용 가능한 Gradle 작업 목록 확인
84	./gradlew dependencies	Gradle 의존성 트리 확인
85	./gradlew build --stacktrace	빌드 실패 시 상세 예외 추적 출력
86	mariadb --version	MariaDB Client 설치/버전 확인
87	mariadb -h mariadb -P 3306 -u devuser -p	app 컨테이너에서 MariaDB 접속
88	mariadb -h 127.0.0.1 -P 3307 -u devuser -p	호스트 PC에서 MariaDB 접속
89	SHOW DATABASES;	MariaDB 데이터베이스 목록 확인
90	USE wizdb;	wizdb 데이터베이스 선택
91	SHOW TABLES;	현재 DB의 테이블 목록 확인
92	SELECT VERSION();	MariaDB 서버 버전 확인

No.	명령어	설명
93	SELECT USER();	현재 MariaDB 접속 사용자 확인
94	node --version	Node.js 버전 확인 - Codex CLI 환경 점검
95	npm --version	npm 버전 확인
96	codex --help	Codex CLI의 현재 지원 옵션/명령 확인
97	codex --version	설치된 Codex CLI 버전 확인
98	codex	Codex AI 대화형 실행
99	env | grep SPRING	Spring Boot 관련 환경변수 확인
100	env | grep OPENAI	OpenAI 관련 환경변수 존재 여부 확인 - 키 값 노출 주의

2. 가장 많이 쓰는 기본 순서

- docker compose up -d
- docker compose ps
- ssh -p 2222 devuser@127.0.0.1
- cd /workspace
- codex
- ./gradlew clean build
- ./gradlew bootRun

3. 장애 발생 시 우선 확인

컨테이너 상태 → 로그 → 포트 → DB 통신 → Gradle 빌드 순으로 확인하면 문제 범위를 빠르게 좁힐 수 있음.

- docker compose ps
- docker compose logs -f app
- docker compose logs -f mariadb
- ss -lntp
- ping mariadb
- ./gradlew build --stacktrace

4. 주의사항

`docker compose down -v`, `docker volume prune`, `docker system prune` 등 삭제/정리 명령은 MariaDB 데이터나 필요한 Docker 리소스를 삭제할 수 있으므로 실행 전에 반드시 백업과 대상을 확인함. `env | grep OPENAI`는 환경에 따라 API 키 값까지 화면에 표시될 수 있으므로 화면 공유·로그 저장 시 주의함.

부록 · Docker 샌드박스 · Compose

Docker Sandboxes YML 실전 예제 20선

Codex CLI + SSH Server + MariaDB + Spring Boot + Kotlin + Rust

본 문서는 20개 sbxenv.yaml 예제를 Word 교본 형태로 정리한 자료임. 각 예제는 Linux 기반 Docker Sandbox에서 Codex를 중심으로 SSH, Spring Boot/Kotlin, MariaDB, Rust/Axum 개발환경을 구성하는 여러 패턴을 보여줌.

이 문서에서 고친 곳

- 문서 생성 과정의 대화 흔적(「앞서 제작한」)을 지웠습니다.

1. 공통 포트 및 개발환경

- SSH: Host 2222 -> Sandbox 22
- Spring Boot/Kotlin: Host 8080 -> Sandbox 8080
- Rust/Axum 일부 예제: Host 9090 -> Sandbox 9090
- MariaDB: Host 3307 -> Sandbox 3306
- 기본 자원: CPU 4 / RAM 8GB
- Codex: agent: codex
- 프로젝트 영구보존: workspace: ./workspace
- 참고문서: additionalWorkspaces + readOnly
- 개발/테스트/운영: dev / test / prod profile

2. 꼭 알아둘 원칙

sbxenv.yaml은 docker-compose.yml이 아님. ports는 통로를 게시할 뿐 SSH 서버, MariaDB, JDK, Rust 또는 Spring Boot를 자동 설치하지 않음. 반복 사용 환경은 필요한 도구가 들어간 custom Sandbox template을 준비한 뒤 YAML의 sandboxOptions.template에서 재사용하는 방식으로 설계함.

lifecycle 명령은 Sandbox 내부 패키지 설치용이 아니라 호스트 측 준비/백업 작업에 사용함. OpenAI 자격증명과 DB 비밀번호는 YAML에 평문으로 저장하지 않는 것을 원칙으로 함.

3. 서버 최초 준비와 구동 순서

```bash
mkdir -p wiz-sandbox/workspace wiz-sandbox/docs wiz-sandbox/backups
cd wiz-sandbox

sbx secret set openai --oauth

# 기본 Codex Sandbox 생성
sbx create --name wiz-builder --cpus 4 --memory 8g codex ./workspace

# Sandbox 내부 root shell
sbx exec -it -u root wiz-builder bash
```

4. Sandbox 내부 패키지 준비 예

```bash
apt-get update

apt-get install -y \
  openssh-server \
  mariadb-server \
  mariadb-client \
  curl git unzip zip \
  build-essential pkg-config libssl-dev \
  openjdk-21-jdk

mkdir -p /run/sshd

java -version
mariadb --version

# Rust는 일반 사용자 계정에서 rustup 설치를 권장
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

5. Codex / 환경 실행

```bash
# 실제 사용할 YAML을 sbxenv.yaml로 복사
cp 20_master_full_stack.sbxenv.yaml sbxenv.yaml

# 적용 계획
sbx env plan

# Sandbox 생성
sbx env create

# Codex agent 실행/attach
sbx env run

# 내부 Bash
sbx env exec -it -- bash
```

6. 서버 구동법

```bash
# SSH Server
mkdir -p /run/sshd
/usr/sbin/sshd

# MariaDB
service mariadb start

# Spring Boot / Kotlin
cd /workspace
./gradlew clean
./gradlew build
./gradlew bootRun

# Rust / Axum 예: 9090
cd /workspace/rust-api
cargo build
cargo run
```

7. 외부 접속 및 확인

```bash
# SSH
ssh -p 2222 devuser@DOCKER_HOST_IP

# MariaDB
mariadb -h DOCKER_HOST_IP -P 3307 -u devuser -p

# Spring Boot
curl http://DOCKER_HOST_IP:8080

# Rust API
curl http://DOCKER_HOST_IP:9090
```

Windows PowerShell 포트 점검:

```powershell
Test-NetConnection DOCKER_HOST_IP -Port 2222
Test-NetConnection DOCKER_HOST_IP -Port 8080
Test-NetConnection DOCKER_HOST_IP -Port 3307
Test-NetConnection DOCKER_HOST_IP -Port 9090
```

8. Spring Boot Profile

```bash
sbx env run --env-arg profile=dev
sbx env run --env-arg profile=test
sbx env run --env-arg profile=prod
```

9. 상태 확인

```bash
sbx ls
sbx ports wiz-master-20

sbx env exec -- codex --version
sbx env exec -- java -version
sbx env exec -- rustc --version
sbx env exec -- cargo --version
sbx env exec -- mariadb --version
sbx env exec -- bash -lc 'ss -lntp'
```

10. YML 실전 예제 20개

예제 1. 01_full_stack_lan.sbxenv.yaml

전체 스택을 LAN에 공개하는 기본 완전체

```yaml
schemaVersion: "1"
name: wiz-full-01
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 2. 02_full_stack_local_only.sbxenv.yaml

전체 스택을 로컬 PC 중심으로 사용하는 구성

```yaml
schemaVersion: "1"
name: wiz-full-02
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 3. 03_springboot_lan_db_local.sbxenv.yaml

Spring Boot는 LAN, MariaDB는 로컬로 제한하는 구성

```yaml
schemaVersion: "1"
name: wiz-full-03
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 127.0.0.1
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 4. 04_springboot_only_public.sbxenv.yaml

Spring Boot만 LAN에 공개하고 SSH/DB 노출을 줄인 구성

```yaml
schemaVersion: "1"
name: wiz-full-04
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 127.0.0.1
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 127.0.0.1
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 5. 05_dev_profile.sbxenv.yaml

개발 profile과 DEBUG 로그 중심 구성

```yaml
schemaVersion: "1"
name: wiz-dev-05
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  LOG_LEVEL: "DEBUG"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 6. 06_test_profile.sbxenv.yaml

테스트 profile/JPA 테스트 중심 구성

```yaml
schemaVersion: "1"
name: wiz-test-06
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  SPRING_JPA_SHOW_SQL: "true"
  APP_MODE: "test"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
```

```
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 7. 07_prod_check.sbxenv.yaml

운영 profile 사전검증용 구성

```yaml
schemaVersion: "1"
name: wiz-prod-07
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  LOG_LEVEL: "INFO"
  APP_MODE: "prod-check"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
```

```
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 8. 08_rust_api_9090.sbxenv.yaml

Rust/Axum 9090 API를 추가한 구성

```yaml
schemaVersion: "1"
name: wiz-rust-08
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  RUST_LOG: "info"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 9090
    host: 9090
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
```

```
    command: mkdir -p workspace docs backups scripts
    timeout: 1m
preRemove:
  - name: Backup persistent workspace
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 9. 09_kotlin_rust_dual_api.sbxenv.yaml

Kotlin Spring Boot + Rust API를 동시에 사용하는 구성

```yaml
schemaVersion: "1"
name: wiz-dual-09
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  RUST_LOG: "debug"
  KOTLIN_ENV: "dev"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 9090
    host: 9090
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
```

```
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 10. 10_readonly_docs.sbxenv.yaml

Codex 참고문서를 read-only로 제공하는 구성

```yaml
schemaVersion: "1"
name: wiz-docs-10
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 11. 11_private_clone.sbxenv.yaml

```yaml
private clone workspace 패턴
schemaVersion: "1"
name: wiz-clone-11
agent: codex

workspace:
  path: ./workspace
  clone: true

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
```

```
    timeout: 1m
preRemove:
  - name: Backup persistent workspace
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 12. 12_more_memory.sbxenv.yaml

RAM 12GB 대형 개발환경

```yaml
schemaVersion: "1"
name: wiz-big-12
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 12g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 13. 13_small_memory.sbxenv.yaml

CPU 2 / RAM 4GB 경량 개발환경

```yaml
schemaVersion: "1"
name: wiz-small-13
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 2
  memory: 4g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 14. 14_skills_readwrite.sbxenv.yaml

공유 skills를 read-write로 사용하는 구성

```yaml
schemaVersion: "1"
name: wiz-skills-14
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readwrite

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 15. 15_no_shared_skills.sbxenv.yaml

공유 skills를 끈 격리형 구성

```yaml
schemaVersion: "1"
name: wiz-noskills-15
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: off

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 16. 16_ipv4_lan_all.sbxenv.yaml

IPv4 LAN 전체 공개형

```yaml
schemaVersion: "1"
name: wiz-ipv4-16
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 17. 17_ssh_local_app_lan.sbxenv.yaml

SSH는 로컬, 애플리케이션은 LAN 공개형

```yaml
schemaVersion: "1"
name: wiz-sshlocal-17
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 127.0.0.1
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
    timeout: 5m
```

예제 18. 18_backup_heavy.sbxenv.yaml

날짜별 workspace 백업 강화형

```yaml
schemaVersion: "1"
name: wiz-backup-18
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
```

```
    command: tar -czf "backups/${SBX_SANDBOX_NAME}-$(date +%Y%m%d_%H%M%S).tar.gz" workspace
    timeout: 5m
```

예제 19. 19_no_workspace_clone.sbxenv.yaml

주 workspace 없이 agent 중심으로 사용하는 변형

```yaml
schemaVersion: "1"
name: wiz-nowork-19
agent: codex


args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

예제 20. 20_master_full_stack.sbxenv.yaml

Codex + SSH + Spring Boot/Kotlin + Rust + MariaDB 최종 완전체

```yaml
schemaVersion: "1"
name: wiz-master-20
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser
  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"
  RUST_BACKTRACE: "1"
  SERVER_ADDRESS: "0.0.0.0"
  RUST_LOG: "info"
  TZ: "Asia/Seoul"

sandboxOptions:
  template: wiz-dev-template:latest
  pullPolicy: missing
  cpus: 4
  memory: 8g
  skills: readonly

ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 9090
    host: 9090
    hostIP: 0.0.0.0
    protocol: tcp4
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
```

```
  initialize:
    - name: Prepare persistent host folders
      command: mkdir -p workspace docs backups scripts
      timeout: 1m
  preRemove:
    - name: Backup persistent workspace
      command: tar -czf backups/${SBX_SANDBOX_NAME}-workspace.tar.gz workspace
      timeout: 5m
```

11. 영구저장과 백업

`workspace: ./workspace`는 로컬 호스트의 프로젝트 디렉터리를 Sandbox에 연결하는 핵심 설정임. 소스, Gradle 파일, SQL dump 등을 workspace에 저장하면 Sandbox 제거 후에도 호스트에 남길 수 있음.

```bash
# MariaDB 논리 백업 예
sbx env exec -- bash -lc \
  'mariadb-dump -u devuser -p wizdb > /workspace/wizdb_backup.sql'
```

12. Cloud Persistent Volume

```bash
sbx --cloud volume create wiz-dbdata
sbx --cloud volume ls

# Cloud Sandbox 생성 시 volume 연결 예
sbx --cloud create --volume wiz-dbdata:/var/lib/mysql codex
```

Cloud persistent volume은 로컬 workspace와 다른 cloud-only 기능임. DB를 장기 운영할 경우 동시 쓰기와 snapshot 특성을 고려해야 함.

13. 장애진단 빠른 순서

```bash
# 1. Sandbox 목록
sbx ls

# 2. 포트 게시
sbx ports wiz-master-20

# 3. 내부 listen
sbx env exec -- bash -lc 'ss -lntp'

# 4. 프로세스
sbx env exec -- bash -lc "ps aux | grep -E 'sshd|maria|java|gradle'"

# 5. Spring Boot 내부 확인
sbx env exec -- curl -v http://127.0.0.1:8080

# 6. MariaDB 내부 확인
sbx env exec -- mariadb -h 127.0.0.1 -P 3306 -u devuser -p
```

14. 보안 체크

- SSH와 MariaDB를 공인 인터넷에 직접 노출하지 않음.

- LAN 공개가 필요 없으면 hostIP를 생략하거나 127.0.0.1로 제한함.
- OpenAI API 키/DB 비밀번호를 YAML과 Git에 평문 저장하지 않음.
- SSH는 가능하면 public key 인증을 사용함.
- Spring Boot만 외부 공개하고 SSH/DB는 로컬로 제한하는 구성이 더 안전함.

부록 · Docker 샌드박스 · Compose

Docker Sandbox Codex Shell SSH 사용법

Docker Sandboxes Codex · Shell · SSH 접속 사용법

Windows CMD 기준 / 현재 구성: codex-dev, ssh22

1. 현재 Sandbox 확인

Windows CMD를 열고 먼저 Sandbox 상태를 확인함.

```bash
sbx ls
```

2. Codex 개발환경으로 들어가기

기존에 만들어 둔 codex-dev Sandbox로 다시 들어갈 때 사용함.

```bash
sbx run --name codex-dev
```

Codex Sandbox는 C:\Users\[사용자명]\projects\codex-test 작업공간과 연결되어 있음. 종료 후 다시 사용할 때도 같은 명령을 실행함.

3. 일반 Linux Shell 환경으로 들어가기

SSH용으로 만들어 둔 ssh22 Sandbox의 일반 Linux Shell로 들어감.

```bash
sbx run --name ssh22
```

정상적으로 들어가면 다음과 비슷한 프롬프트가 표시됨.

```text
agent@ssh22:linux$
```

4. ssh22에서 SSH 서버 확인 및 실행

ssh22가 새로 시작된 후 SSH 서버가 실행 중인지 확인함.

```bash
sudo ss -lntp | grep :22
sudo mkdir -p /run/sshd
sudo /usr/sbin/sshd
sudo ss -lntp | grep :22
```

첫 번째 확인에서 22번 LISTEN이 이미 보이면 sshd를 다시 실행할 필요는 없음.

5. Windows CMD에서 SSH로 접속하기

현재 ssh22의 포트 매핑은 Windows 127.0.0.1:22 → ssh22:22 형태이므로 Windows 자기 자신에서 localhost로 접속함.

```bash
ssh devuser@localhost
```

비밀번호 입력 후 다음 프롬프트가 나오면 성공임.

```text
devuser@ssh22:~$
```

6. 평소 사용하는 순서

목적	Windows CMD 명령	설명
상태 확인	sbx ls	Codex/Shell Sandbox의 running·stopped 상태 확인
Codex로 이동	sbx run --name codex-dev	AI 코딩용 Codex Sandbox 실행/재접속
Shell로 이동	sbx run --name ssh22	일반 Ubuntu Linux Shell 실행/재접속
SSH 접속	ssh devuser@localhost	ssh22가 실행 중이고 sshd가 동작할 때 접속

7. 꼭 기억할 점

- ssh22가 stopped 상태이면 ssh devuser@localhost만 실행해서는 접속되지 않음.
- 먼저 sbx run --name ssh22로 Sandbox를 실행해야 함.
- 현재 로컬 sbx의 exit는 세션을 종료하면서 Sandbox가 stopped 상태가 될 수 있음.
- 현재 ssh22는 127.0.0.1:22로 publish되어 있으므로 다른 PC에서 직접 접속하는 구성은 아님.
- Codex와 ssh22는 서로 별도의 Sandbox이므로 한쪽에 설치한 프로그램·사용자 설정이 다른 쪽에 자동으로 생기지 않음.

8. 한눈에 보는 명령

```bash
[Codex]
sbx run --name codex-dev

[Shell]
sbx run --name ssh22

[ssh22 내부 – SSH 서버 확인]
sudo ss -lntp | grep :22

[필요할 때 sshd 실행]
sudo mkdir -p /run/sshd
sudo /usr/sbin/sshd

[Windows CMD – SSH 접속]
ssh devuser@localhost
```

부록 · Docker 샌드박스 · Compose

Windows 11 Docker Sandbox + Codex wizdev 개발환경 구축 교본

Docker Sandboxes에서 격리된 Linux 개발환경을 만들고 Codex Agent로 프로그램을 개발·빌드·테스트하는 기본 절차

1. 전체 구조

```powershell
Windows 11 실제 PC
  └ C:\Users\[사용자명]\wizdev  : 실제 프로젝트 소스 폴더
       ↕ workspace 연결
    Docker Sandbox : wizdev
      ├ Ubuntu Linux 격리 개발환경
      ├ Python / Git / curl 등 개발도구
      └ Codex Agent
           ├ 소스코드 생성·수정
           ├ 빌드·실행
           ├ 테스트
           └ 오류 분석 및 재수정
```

2. wizdev 생성 및 Codex 실행 순서

순서	실행 위치	명령/작업	설명
1	PowerShell	mkdir C:\Users\[사용자명]\wizdev	Windows에 실제 개발 프로젝트 폴더를 생성함
2	PowerShell	cd C:\Users\[사용자명]\wizdev	wizdev 폴더로 이동함
3	PowerShell	sbx create --name wizdev codex .	현재 폴더(.)를 연결한 Codex용 Docker Sandbox를 생성함
4	PowerShell	sbx ls	생성된 Sandbox 목록과 상태를 확인함
5	PowerShell	sbx run --name wizdev	wizdev Sandbox를 시작하고 Codex Agent에 접속함
6	Codex	계정 로그인/인증	최초 실행 시 사용하는 OpenAI/Codex 계정으로 인증함
7	Codex 프롬프트	자연어 개발 요청 입력	원하는 프로그램의 기술스택과 기능을 한국어로 지시함
8	Codex	자동 개발	파일 생성 → 코드 작성 → 빌드 → 테스트 → 오류 수정 과정을 수행함
9	종료	exit 또는 Agent 종료	Sandbox/Agent에서 빠져나와 PowerShell로 돌아감
10	재사용	sbx run --name wizdev	다음 작업 때 기존 wizdev 개발환경을 다시 사용함

3. 처음 실행할 명령

Windows PowerShell에서 다음 순서로 실행함.

```powershell
mkdir C:\Users\[사용자명]\wizdev
cd C:\Users\[사용자명]\wizdev
sbx create --name wizdev codex .

sbx ls
sbx run --name wizdev
```

4. Codex에게 개발시키는 예제 프롬프트

Kotlin + Spring Boot로 회원관리 REST API 프로젝트를 만들어줘.
조건:
- Kotlin
- Spring Boot
- Gradle
- JPA
- DTO
- Entity
- Repository
- Service
- Controller
- MariaDB
- 회원 등록/조회/수정/삭제 CRUD
- 테스트 코드 포함
현재 workspace에 실제 프로젝트 파일을 생성하고,
빌드와 테스트까지 실행해줘.
오류가 발생하면 원인을 분석해서 수정한 후 다시 테스트해줘.

5. 핵심 개념

Docker Sandbox : Windows PC 내부에 격리된 Linux 개발환경을 만들어 줌. 개발·실행·테스트용 가상 개발 서버처럼 이해하면 쉬움.

wizdev 폴더 :

> `C:\Users\[사용자명]\wizdev에 위치하는 실제 프로젝트 소스 폴더임. Sandbox와 연결하여 Codex가 파일을 생 powershell 성·수정함.`

Codex Agent : Sandbox 내부에서 자연어 지시를 받아 코드를 작성하고 명령을 실행하며 테스트와 오류 수정까지 수행하는 AI 코딩 Agent임.

테스트 환경 : Sandbox 내부에서 빌드, 실행, 단위 테스트, REST API 테스트 등을 수행할 수 있어 Windows 개발환경과 격리할 수 있음.

6. 작업 흐름

개발 요청 → Codex가 파일 생성 → 코드 작성 → 빌드 → 실행 → 테스트 → 오류 확인 → 코드 수정 → 재빌드·재테스트 → 완성

7. 주의사항

- Sandbox를 생성할 때 마지막의 점(.)은 현재 Windows 프로젝트 폴더를 workspace로 연결한다는 의미이므로 생략하지 않는 것이 좋음.
- shell Agent로 생성한 Sandbox에는 Codex가 자동 포함되지 않으므로 Codex 개발용은 'sbx create ... codex .' 방식으로 별도 생성함.

- 네트워크 접근은 Sandbox 정책에 의해 제한될 수 있음. 외부 Ollama, MariaDB, API 서버 등을 사용할 때 필요한 도메인·포트를 정책에서 허용해야 할 수 있음.
- 프로젝트 소스는 Git으로 버전 관리하여 AI Agent의 대규모 수정 전후를 쉽게 비교·복구하도록 하는 것이 좋음.

부록 · Docker 샌드박스 · Compose

Docker Sandbox Codex SpringBoot Kotlin MariaDB SSH 초보자교본 (1)

Docker Sandbox + Codex AI Spring Boot Kotlin + MariaDB + SSH 초보자 실습 교본

YAML 중심의 통합 개발환경 구축 · 접속 · 빌드 · 실행 · DB 확인

이 교본은 Docker 기반 격리 개발환경에서 Kotlin + Spring Boot 애플리케이션, MariaDB, SSH 접속환경을 함께 구성하고, Codex AI를 이용해 코드를 수정한 뒤 clean → build → 실행하는 전체 흐름을 초보자 관점에서 정리함.

1. 완성 후 전체 구조

```bash
Windows / Linux 호스트
        |
        +-- SSH 2222 ----------> app 컨테이너 :22
        |
        +-- HTTP 8080 ---------> Spring Boot :8080
        |
        +-- DB 3307 -----------> MariaDB :3306
                                  ^
                                  |
                        mariadb:3306
                                  |
                        Spring Boot Kotlin
                                  |
                             Codex AI
```

핵심은 호스트에서 MariaDB에 접속할 때는 127.0.0.1:3307을 사용하고, app 컨테이너의 Spring Boot에서는 Docker 서비스 이름인 mariadb:3306을 사용한다는 점임.

2. 준비물

항목	용도
Docker Desktop 또는 Docker Engine	컨테이너 실행
Docker Compose	compose.yml 실행
SSH Client	가상 개발환경 접속
JDK 21	Spring Boot/Kotlin 실행
Gradle Wrapper	빌드
Codex AI	프로젝트 분석·코드 수정·명령 실행 지원
MariaDB	애플리케이션 데이터베이스

3. 프로젝트 폴더

```text
kotlin-sandbox/
├── compose.yml
└── app/
    ├── build.gradle.kts
    ├── settings.gradle.kts
    ├── gradlew
    ├── gradlew.bat
    ├── gradle/
    └── src/
        └── main/
            ├── kotlin/
            └── resources/
                └── application.yml
```

4. 전체 compose.yml

아래 예제는 학습용임. 실제 운영환경에서는 비밀번호를 YAML에 직접 기록하지 말고 .env 또는 secrets로 분리하는 것이 좋음.

```yaml
services:
  mariadb:
    image: mariadb:11
    container_name: sbx-mariadb
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: root1234
      MARIADB_DATABASE: wizdb
      MARIADB_USER: devuser
      MARIADB_PASSWORD: CHANGE_ME
    ports:
      - "3307:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mariadb-admin", "ping", "-h", "127.0.0.1",
             "-uroot", "-proot1234"]
      interval: 10s
      timeout: 5s
      retries: 10

  app:
    image: eclipse-temurin:21-jdk
    container_name: sbx-kotlin
    restart: unless-stopped
    working_dir: /workspace
    depends_on:
      mariadb:
        condition: service_healthy
    ports:
      - "8080:8080"
      - "2222:22"
    volumes:
      - ./app:/workspace
      - gradle_cache:/root/.gradle
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/wizdb
      SPRING_DATASOURCE_USERNAME: devuser
      SPRING_DATASOURCE_PASSWORD: CHANGE_ME
    command:
      - /bin/bash
      - -c
      - |
        apt-get update
        apt-get install -y openssh-server curl git vim nano \
          mariadb-client iputils-ping net-tools procps

        mkdir -p /run/sshd

        if ! id devuser >/dev/null 2>&1; then
          useradd -m -s /bin/bash devuser
        fi

        echo 'devuser:CHANGE_ME' | chpasswd

        sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' \
          /etc/ssh/sshd_config
```

```
        sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication yes/' \
          /etc/ssh/sshd_config

        chown -R devuser:devuser /workspace || true
        /usr/sbin/sshd

        echo "Docker Kotlin Sandbox Ready"
        echo "SSH       : localhost:2222"
        echo "SpringBoot: localhost:8080"
        echo "MariaDB   : mariadb:3306"

        tail -f /dev/null

volumes:
  mariadb_data:
  gradle_cache:
```

5. 실행과 상태 확인

```bash
cd kotlin-sandbox
docker compose up -d
docker compose ps
```

문제가 있으면 로그를 확인함.

```bash
docker compose logs -f app
docker compose logs -f mariadb
```

6. SSH로 가상환경 접속

Docker가 실행되는 같은 PC에서 접속:

```bash
ssh -p 2222 devuser@127.0.0.1
```

비밀번호: CHANGE_ME

다른 LAN PC에서 Docker 호스트의 IP가 SERVER_IP인 경우:

```bash
ssh -p 2222 devuser@SERVER_IP
```

접속 후 확인:

```bash
whoami
pwd
java -version
cd /workspace
ls
```

7. Spring Boot application.yml

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mariadb://mariadb:3306/wizdb
    username: devuser
    password: CHANGE_ME
    driver-class-name: org.mariadb.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

컨테이너 내부에서 localhost:3307을 사용하면 안 됨. Spring Boot와 MariaDB는 Docker 내부 네트워크에서 서비스명 mariadb와 내부 포트 3306으로 통신함.

8. build.gradle.kts 핵심 의존성

```kotlin
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

9. MariaDB 직접 접속

호스트 PC에서:

```bash
mariadb -h 127.0.0.1 -P 3307 -u devuser -p
```

비밀번호: CHANGE_ME

app 컨테이너에서:

```bash
mariadb -h mariadb -P 3306 -u devuser -p
```

DB 확인 예:

```sql
SHOW DATABASES;
USE wizdb;
SHOW TABLES;
```

10. Spring Boot clean → build → 실행

```bash
ssh -p 2222 devuser@127.0.0.1
cd /workspace

chmod +x gradlew
./gradlew clean
./gradlew build
./gradlew bootRun
```

브라우저 또는 REST 클라이언트에서는 http://127.0.0.1:8080 으로 접근함.

11. Codex AI를 함께 사용하는 흐름

Codex AI는 프로젝트 폴더를 기준으로 소스 구조를 읽고 수정하거나, 빌드 오류를 분석하고 필요한 변경을 제안·적용하는 개발 보조 도구로 활용할 수 있음. Codex의 정확한 실행 명령과 인증 방식은 설치 버전에 따라 달라질 수 있으므로 현재 설치된 Codex CLI의 도움말을 먼저 확인하는 방식이 안전함.

```bash
cd /workspace

codex --help
codex
```

권장 작업 순서:

```text
1) SSH로 sbx-kotlin 접속
2) cd /workspace
3) Codex 실행
4) "Spring Boot Kotlin 프로젝트를 분석해줘"
5) "MariaDB JPA 연결 설정을 확인해줘"
6) "필요한 코드를 수정해줘"
7) Codex 종료 또는 별도 터미널 사용
8) ./gradlew clean
9) ./gradlew build
10) ./gradlew bootRun
11) 브라우저/REST API로 테스트
12) MariaDB에서 저장 결과 확인
```

Codex에게 전달할 초보자용 예시 지시문:

```text
이 프로젝트는 Kotlin + Spring Boot + JPA + MariaDB 프로젝트다.
DB 서비스명은 mariadb, 내부 포트는 3306이다.
기존 코드를 먼저 분석하고 오류가 있으면 수정해줘.
수정 후 ./gradlew clean 과 ./gradlew build 로 검증해줘.
실패하면 오류 원인을 설명하고 다시 수정해줘.
```

12. Codex까지 컨테이너에서 사용하려면

현재 compose.yml의 app 컨테이너는 JDK 중심 이미지이므로 Codex CLI가 기본 포함되어 있지는 않음. Codex CLI를 컨테이너 안에서 직접 실행하려면 해당 CLI가 요구하는 런타임/설치 절차와 인증정보를 추가해야 함. 반대로 호스트의 Codex에서 ./app 폴더를 열어 수정하고, SSH 터미널에서는 빌드·실행만 하는 구성도 단순하고 관리하기 쉬움.

즉 초보자에게는 다음 두 방식 중 ①을 먼저 권장함.

```text
① 호스트 Codex → ./app 소스 수정
   SSH → /workspace에서 clean/build/bootRun

② SSH → 컨테이너 내부 Codex 실행
   → 코드 수정 → clean/build/bootRun
   (Codex CLI 설치 및 인증 설정이 추가로 필요)
```

13. 포트 한눈에 보기

기능	호스트 포트	컨테이너 포트	접속 예
SSH	2222	22	ssh -p 2222 devuser@127.0.0.1
Spring Boot	8080	8080	http://127.0.0.1:8080
MariaDB	3307	3306	mariadb -h 127.0.0.1 -P 3307 ...

14. 자주 발생하는 오류

SSH 접속 거부:

```bash
docker compose ps로 app 실행 여부를 확인하고 docker compose logs app 및 컨테이너 내부 sshd 상태를 확인함.
```

DB 연결 실패: Spring Boot의 JDBC 주소가 jdbc:mariadb://mariadb:3306/wizdb인지 확인함. 컨테이너 내부에서 127.0.0.1:3307을 사용하지 않음.

gradlew 실행 권한 오류:

```bash
chmod +x gradlew 실행 후 다시 ./gradlew build를 실행함.
```

8080 포트 충돌: 호스트에서 이미 8080을 사용 중이면 compose.yml의 왼쪽 포트만 예: 8081:8080으로 변경함.

3307 포트 충돌: 호스트 포트를 예: 3308:3306으로 변경하고 호스트 DB 접속 시 -P 3308을 사용함.

15. 종료·재시작·초기화

```bash
# 종료
docker compose down

# 다시 시작
docker compose up -d

# 컨테이너 상태
docker compose ps

# DB 데이터까지 완전히 삭제하고 초기화
docker compose down -v
```

주의:

```bash
docker compose down -v는 MariaDB 볼륨 데이터까지 삭제하므로 필요한 DB가 있으면 실행하지 않음.
```

16. 최종 실습 체크리스트

- Docker 및 Compose가 정상 설치됨
- compose.yml로 MariaDB와 app 컨테이너가 실행됨
- ssh -p 2222 devuser@127.0.0.1 접속 성공
- /workspace에 Kotlin Spring Boot 프로젝트가 보임
- MariaDB 서비스명 mariadb:3306으로 연결됨
- ./gradlew clean 성공
- ./gradlew build 성공
- ./gradlew bootRun 성공
- 127.0.0.1:8080에서 API 접근 성공
- Codex AI로 소스 분석/수정 후 다시 빌드 검증함

17. 핵심 명령어 요약

```bash
docker compose up -d
docker compose ps

ssh -p 2222 devuser@127.0.0.1

cd /workspace
./gradlew clean
./gradlew build
./gradlew bootRun

mariadb -h 127.0.0.1 -P 3307 -u devuser -p

docker compose logs -f app
docker compose down
```

끝 - Docker + Kotlin + Spring Boot + MariaDB + SSH + Codex AI 통합 실습

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · Docker 샌드박스 · Compose

Docker Sandboxes 완전 초보 실전 구축 교본

Linux 가상환경 + Codex AI + SSH + Kotlin/Spring Boot + MariaDB + 외부접속 + 영구저장

Docker Sandboxes 0.43.0 계열 기준 · sbxenv.yaml 실전 확장판

이 교본은 Docker Sandboxes의 `sbx`와 실험적 `sbx env`/`sbxenv.yaml`을 이용하여 AI 코딩 환경을 설계하는 방법을 초보자 관점에서 설명함. Docker Compose와 sbxenv.yaml의 역할을 구분하고, 실제 외부 접속과 영구 보존에서 자주 생기는 오해를 바로잡는 데 중점을 둠.

이 문서에서 고친 곳

- 문서 생성 과정의 대화 흔적(「요청한」)을 지웠습니다.

0. 가장 중요한 결론부터

- sbxenv.yaml은 docker-compose.yml이 아님. `services: mariadb:`처럼 여러 컨테이너 서비스를 선언하는 파일이 아님.
- `ports`는 포트를 연결할 뿐 SSH 서버, MariaDB 서버, Spring Boot를 자동 설치하거나 실행하지 않음.
- `lifecycle`은 Sandbox 내부가 아니라 호스트에서 실행되는 명령이므로 Sandbox 내부 apt 설치용으로 사용하면 안 됨.
- 로컬 개발에서는 `workspace: ./workspace`가 호스트 디렉터리를 Sandbox에 연결하므로 프로젝트 소스 영구 보존의 핵심임.
- Cloud persistent volume은 cloud-only이며 Sandbox 종료 시 snapshot으로 저장됨.
- SSH와 MariaDB를 LAN에 공개할 때는 `hostIP: 0.0.0.0`뿐 아니라 호스트 방화벽, 서비스 listen 주소, 계정 보안까지 함께 설정해야 함.
- Codex 자격증명은 YAML에 API 키를 평문으로 넣기보다 Docker Sandboxes Secret/OAuth 기능을 우선 사용함.

1. 목표 아키텍처

```text
외부 Windows / Linux PC
          |
          | LAN
          ▼
Docker Host (예: SERVER_IP)
          |
          ├── TCP 2222 ──────> Sandbox TCP 22
          |                     SSH Server
          |
          ├── TCP 8080 ──────> Sandbox TCP 8080
          |                     Kotlin + Spring Boot
          |
          └── TCP 3307 ──────> Sandbox TCP 3306
                                MariaDB

Sandbox 내부 개발 흐름
Codex AI
   |
   ├── Kotlin 코드 분석/수정
   ├── Gradle clean/build/test
   └── Spring Boot 실행
            |
            ▼
        JPA / JDBC
            |
            ▼
         MariaDB
```

이 구조의 핵심은 'Sandbox 자체'와 '그 안에서 실행되는 서비스'를 구분하는 것임. sbxenv.yaml은 Sandbox의 agent, workspace, 포트, 자원, 환경변수 등을 선언함. SSH/MariaDB/JDK 같은 내부 도구를 반복 가능하게 준비하려면 적절한 Sandbox template 또는 kit을 만들거나, 생성 후 `sbx exec`로 내부 설치 과정을 수행해야 함.

2. 권장 프로젝트 디렉터리

```text
wiz-sandbox/
├── sbxenv.yaml              # Sandbox 환경 선언
├── workspace/               # 영구 보존되는 주 작업 폴더
│   ├── build.gradle.kts
│   ├── settings.gradle.kts
│   ├── gradlew
│   ├── gradle/
│   └── src/
├── docs/                    # Codex 참고문서(read-only 권장)
├── backups/                 # 삭제 전 백업
├── config/
│   ├── ssh/
│   ├── mariadb/
│   └── spring/
└── scripts/
    ├── sandbox-install.sh
    ├── sandbox-start.sh
    ├── db-init.sql
    ├── backup.sh
    └── check.sh
```

환경 파일은 직접 마운트되는 workspace 바깥에 두는 것이 권장됨. 위 구조에서는 `sbxenv.yaml`과 `workspace/`가 형제 디렉터리이므로 관리하기 편함.

3. sbxenv.yaml 완전체 기준안

```yaml
schemaVersion: "1"

name: wiz-full-dev
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}

  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser

  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"

sandboxOptions:
  cpus: 4
  memory: 8g

ports:
  # SSH
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4

  # Spring Boot
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4

  # MariaDB
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
```

```
    - name: Prepare host directories
      command: |
        mkdir -p workspace
        mkdir -p docs
        mkdir -p backups
        mkdir -p config/ssh
        mkdir -p config/mariadb
        mkdir -p config/spring
        mkdir -p scripts
      timeout: 1m

  preRemove:
    - name: Backup workspace before removal
      command: |
        mkdir -p backups
        tar -czf backups/wiz-workspace-backup.tar.gz workspace
      timeout: 5m
```

이 파일은 Sandbox의 외형과 연결을 선언하는 기준안임. 이 YAML만으로 Sandbox 내부에 sshd와 MariaDB가 설치되는 것은 아님. 다음 장에서 내부 서비스 준비를 별도로 설명함.

4. YAML 항목 한 줄씩 이해하기

항목	설명
schemaVersion	현재 환경 파일 스키마 버전. 지원되지 않는 필드는 loader가 거부할 수 있음.
name	Sandbox 이름. `--name`으로 실행 시 일시적으로 override 가능.
agent: codex	Sandbox에서 사용할 AI coding agent를 Codex로 선택.
workspace	주 프로젝트 디렉터리를 Sandbox workspace로 mount.
additionalWorkspaces	추가 코드/문서 디렉터리를 mount.
readOnly: true	Codex가 참고는 가능하지만 해당 추가 workspace 수정은 막음.
args.profile	dev/test/prod 중 실행 시 선택 가능한 사용자 인자.
env	Sandbox agent 세션에 전달할 환경변수.
sandboxOptions.cpus	Sandbox CPU 수.
sandboxOptions.memory	Sandbox 메모리 제한.
ports.sandbox	Sandbox 내부 서비스 포트.
ports.host	Docker Host에서 사용할 포트.
ports.hostIP	0.0.0.0이면 호스트의 IPv4 인터페이스에 게시. LAN 공개 효과가 있음.
protocol	tcp4로 IPv4 TCP 게시.
lifecycle.initialize	Sandbox 내부가 아닌 호스트에서 환경 적용 전에 실행되는 명령.
lifecycle.preRemove	Sandbox 제거 전에 호스트에서 실행되는 명령.

5. CPU 4 / RAM 8GB

```yaml
sandboxOptions:
  cpus: 4
  memory: 8g
```

Spring Boot, Gradle, Codex, MariaDB를 한 Sandbox에서 학습/개발 목적으로 함께 사용한다면 4 CPU와 8GB는 무난한 출발점임. 대형 프로젝트 빌드나 많은 테스트가 있으면 메모리를 늘릴 수 있음. 환경 파일에서 자원 설정을 변경해도 기존 Sandbox에 즉시 적용되지 않을 수 있으므로 제거 후 재생성이 필요할 수 있음.

6. Codex AI 기본 설정

Codex는 `agent: codex`로 지정함. OpenAI 자격증명은 YAML에 키를 직접 적는 방식보다 Docker Sandboxes Secret/OAuth를 우선 사용함.

```bash
sbx secret set openai --oauth
sbx secret ls

cd wiz-sandbox
sbx env plan
sbx env run
```

OAuth 방식은 OpenAI Secret을 저장하고 Sandbox proxy가 agent의 API 요청을 인증하는 구조로 사용할 수 있음. Secret 값 자체를 프로젝트 YAML에 넣지 않는 것이 핵심임.

7. Codex에게 실제로 시킬 작업 20개

1. 현재 Kotlin + Spring Boot 프로젝트의 전체 폴더 구조를 분석하고 개선점을 설명해줘.
2. build.gradle.kts를 확인하고 Spring Web, JPA, MariaDB 의존성이 올바른지 점검해줘.
3. Member Entity와 Repository를 만들어줘.
4. Service 계층을 추가하고 Controller가 Repository를 직접 호출하지 않도록 수정해줘.
5. REST API의 요청/응답 DTO를 추가해줘.
6. MariaDB 연결 설정을 application.yml과 profile별 파일로 분리해줘.
7. 개발 profile에서 Hibernate ddl-auto=update가 적용되도록 해줘.
8. 운영 profile에서는 ddl-auto를 안전한 값으로 변경해줘.
9. GET /api/members API를 만들고 테스트 코드를 작성해줘.
10. POST /api/members 입력 검증을 추가해줘.
11. 예외 처리를 @RestControllerAdvice 방식으로 정리해줘.
12. 현재 프로젝트를 ./gradlew clean build로 검증해줘.
13. 빌드 오류가 있으면 원인을 분석하고 수정한 뒤 다시 build해줘.
14. Spring Boot가 8080에서 정상 기동되는지 확인해줘.
15. MariaDB 연결 오류가 있으면 host, port, username 설정을 점검해줘.
16. JPA SQL 로그가 개발 profile에서만 보이게 해줘.

17. 프로젝트 README에 실행 방법을 초보자용으로 작성해줘.

18. 보안상 소스에 하드코딩된 비밀번호가 있는지 찾아줘.

19. Git에 커밋하면 안 되는 파일을 찾아 .gitignore를 보완해줘.

20. 수정이 끝나면 변경 파일 목록과 실행 명령을 요약해줘.

8. SSH 2222 → 22

```yaml
ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
```

이 설정은 '포트 통로'만 만듦. Sandbox 내부 TCP 22에서 실제 sshd가 listen 중이어야 함. 또한 Sandbox 내부 사용자의 비밀번호 또는 SSH public key 인증이 준비되어 있어야 함.

9. SSH 서버 준비 개념

반복 가능한 환경에서는 SSH 서버가 포함된 custom template/kit을 만드는 것이 이상적임. 학습 단계에서는 Sandbox 생성 후 root 권한으로 내부 패키지를 설치하는 방식도 가능함. 실제 명령은 template의 Linux 배포판에 따라 달라짐.

```bash
sbx env create
sbx exec -it -u root wiz-full-dev bash

# Sandbox 내부가 Debian/Ubuntu 계열인 경우의 예
apt-get update
apt-get install -y openssh-server
mkdir -p /run/sshd

# sshd 설정과 사용자/키 구성 후
/usr/sbin/sshd
```

systemd가 없는 이미지에서는 `systemctl start ssh`가 동작하지 않을 수 있으므로 `/usr/sbin/sshd` 직접 실행 또는 이미지의 init 방식에 맞춘 실행이 필요함.

10. SSH 접속 방법

```bash
# Docker Host 자체에서
ssh -p 2222 devuser@127.0.0.1

# LAN 다른 PC에서
ssh -p 2222 devuser@SERVER_IP
```

SERVER_IP은 예시 Docker Host IP임. 실제 Host IP를 사용해야 함.

11. Windows에서 SSH 포트 검사

```powershell
Test-NetConnection SERVER_IP -Port 2222
```

TcpTestSucceeded가 True이면 Host까지 포트 연결은 가능하다는 뜻임. SSH 로그인 실패는 그 다음 단계인 sshd, 사용자, 키/비밀번호, 방화벽 설정을 확인함.

12. Spring Boot 8080 → 8080

```yaml
ports:
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
```

Spring Boot가 Sandbox 내부에서 127.0.0.1에만 bind되어 있으면 외부 게시가 기대대로 동작하지 않을 수 있음.
개발용으로는 서버 주소를 적절히 설정하고 `ss -lntp`로 실제 listen 상태를 확인함.

```bash
sbx env exec -- ./gradlew bootRun
sbx env exec -- bash -lc "ss -lntp | grep 8080 || true"
```

13. Spring Boot 기본 프로젝트 구성

```text
workspace/
├── build.gradle.kts
├── settings.gradle.kts
├── gradlew
├── gradle/
└── src/
    ├── main/
    │   ├── kotlin/com/wiz/app/
    │   │   ├── Application.kt
    │   │   ├── controller/
    │   │   ├── service/
    │   │   ├── repository/
    │   │   ├── entity/
    │   │   └── dto/
    │   └── resources/
    │       ├── application.yml
    │       ├── application-dev.yml
    │       ├── application-test.yml
    │       └── application-prod.yml
    └── test/
```

14. build.gradle.kts 기본 예

```kotlin
plugins {
    kotlin("jvm") version "2.2.20"
    kotlin("plugin.spring") version "2.2.20"
    kotlin("plugin.jpa") version "2.2.20"
    id("org.springframework.boot") version "3.5.6"
    id("io.spring.dependency-management") version "1.1.7"
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

15. Spring Profile dev/test/prod

```bash
args:
  profile:
    default: dev
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
# 개발
sbx env run --env-arg profile=dev

# 테스트용 별도 Sandbox 이름
sbx env create --name wiz-test --env-arg profile=test
sbx env exec --name wiz-test -- ./gradlew test

# 운영 profile 검증
sbx env create --name wiz-prod-check --env-arg profile=prod
```

16. application.yml

```yaml
server:
  port: 8080

spring:
  application:
    name: wiz-sandbox-app
```

17. application-dev.yml

```yaml
spring:
  datasource:
    url: jdbc:mariadb://${DB_HOST:127.0.0.1}:${DB_PORT:3306}/${DB_NAME:wizdb}
    username: ${DB_USER:devuser}
    password: ${DB_PASSWORD:}

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

DB_PASSWORD를 소스에 고정값으로 넣지 않는 것이 중요함. 애플리케이션 자체 DB 비밀번호 전달 전략은 별도로 설계해야 하며, Docker Sandboxes의 agent service secret과 애플리케이션 환경변수는 같은 개념이 아님.

18. MariaDB 3307 → 3306

```yaml
ports:
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4
```

이 역시 포트 게시만 수행함. MariaDB 서버가 Sandbox 내부 3306에서 실행되어야 하며, 외부 접속을 허용하는 bind-address와 사용자 host 권한도 별도로 확인해야 함.

19. MariaDB 설치 예

```bash
sbx exec -it -u root wiz-full-dev bash

# Debian/Ubuntu 계열 예
apt-get update
apt-get install -y mariadb-server mariadb-client

# 이미지에 systemd가 없다면 mariadbd를 이미지 방식에 맞게 실행
mariadbd --version
```

20. MariaDB 초기화 SQL 예

```sql
CREATE DATABASE IF NOT EXISTS wizdb
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'devuser'@'%'
  IDENTIFIED BY 'CHANGE_ME_STRONG_PASSWORD';

GRANT ALL PRIVILEGES ON wizdb.*
  TO 'devuser'@'%';

FLUSH PRIVILEGES;
```

문서의 비밀번호는 예시 placeholder임. 실제 비밀번호는 별도 Secret/보안 저장소에서 관리하고 Git에 커밋하지 않음.

21. MariaDB 외부 접속

```bash
# Docker Host
mariadb -h 127.0.0.1 -P 3307 -u devuser -p

# LAN 다른 PC
mariadb -h SERVER_IP -P 3307 -u devuser -p
```

22. Windows 포트 점검

```powershell
Test-NetConnection SERVER_IP -Port 2222
Test-NetConnection SERVER_IP -Port 8080
Test-NetConnection SERVER_IP -Port 3307
```

Host 포트	Sandbox 포트	서비스	접속 예
2222	22	SSH	ssh -p 2222 devuser@HOST_IP
8080	8080	Spring Boot	http://HOST_IP:8080
3307	3306	MariaDB	mariadb -h HOST_IP -P 3307 -u devuser -p

23. hostIP: 0.0.0.0의 의미

hostIP를 생략하면 Docker Sandboxes의 로컬 포트 게시 기본은 loopback 중심임. LAN의 다른 장치에서 접근시키려면 환경 파일에서 명시적으로 hostIP를 설정할 수 있음. 0.0.0.0은 편리하지만 노출 범위가 넓어지므로 SSH와 DB는 특히 주의해야 함.

인터넷 공유기에서 추가 포트포워딩까지 하면 공인 인터넷에 노출될 수 있으므로 개발환경의 SSH/MariaDB 직접 공개는 권장하지 않음.

24. 영구 workspace

```yaml
workspace: ./workspace
```

로컬 Sandbox에서 가장 중요한 영구 보존 수단임. 프로젝트 파일을 Sandbox 내부 임의 디렉터리에만 저장하지 말고 workspace에 두면 Sandbox가 없어져도 호스트의 `./workspace` 파일은 유지됨.

- Spring Boot 소스코드
- Gradle 설정
- SQL 초기화 파일
- README
- 테스트 코드
- 프로젝트별 문서

25. 추가 read-only workspace

```yaml
additionalWorkspaces:
  - path: ./docs
    readOnly: true
```

법률 문서, 설계서, API 명세, 회사 규칙처럼 Codex가 참고는 해야 하지만 수정하면 안 되는 자료를 넣기에 적합함.

26. 삭제 전 자동 백업

```yaml
lifecycle:
  preRemove:
    - name: Backup workspace before removal
      command: |
        mkdir -p backups
        tar -czf backups/wiz-workspace-backup.tar.gz workspace
      timeout: 5m
```

중요: lifecycle은 호스트에서 실행됨. 따라서 이 백업은 오히려 적절한 lifecycle 사용 사례임. Sandbox 내부 DB 메모리 상태를 자동으로 일관성 있게 백업해주는 것은 아니므로 MariaDB는 mysqldump 같은 DB 전용 백업도 별도로 고려함.

27. 날짜가 들어간 백업 스크립트

```bash
#!/bin/bash
set -e

mkdir -p backups
STAMP=$(date +%Y%m%d_%H%M%S)

tar -czf "backups/workspace_${STAMP}.tar.gz" workspace

echo "Backup completed: backups/workspace_${STAMP}.tar.gz"
```

28. MariaDB 논리 백업

```bash
# Sandbox 내부에서 예
mariadb-dump -u devuser -p wizdb > /workspace/wizdb_backup.sql
```

DB dump를 workspace 아래에 두면 로컬 호스트에도 보존됨. 운영 데이터는 보안·암호화·접근권한을 별도로 고려해야 함.

29. Cloud persistent volume

Cloud Sandbox의 persistent volume은 로컬 workspace와 다른 기능임. Cloud-only이며 데이터는 Sandbox가 종료될 때 snapshot으로 저장됨. 여러 Sandbox가 같은 volume을 동시에 mount하면 마지막 종료 Sandbox의 snapshot이 이전 snapshot을 덮어쓸 수 있으므로 DB 동시 쓰기 용도로는 주의해야 함.

```bash
sbx --cloud volume create wiz-dbdata
sbx --cloud volume ls
sbx --cloud volume inspect wiz-dbdata

# Cloud Sandbox 생성 시 기존 volume 연결 예
sbx --cloud create --volume wiz-dbdata:/var/lib/mysql codex
```

30. Cloud volume과 로컬 workspace 비교

구분	로컬 workspace	Cloud persistent volume
용도	소스/프로젝트 파일 호스트 보존	Cloud Sandbox 간 데이터 보존
위치	호스트 디렉터리 bind mount	Docker Cloud volume
보존	호스트 파일이 그대로 남음	종료 시 snapshot
DB 사용	설계 가능하나 권한/성능 검증 필요	동시 mount 시 last-exit-wins 주의
명령	workspace: ./workspace	sbx --cloud volume ...

31. 최초 구축 순서

1. 프로젝트 루트 `wiz-sandbox` 생성
2. `workspace`, `docs`, `backups`, `scripts` 생성
3. `sbxenv.yaml` 작성
4. `sbx secret set openai --oauth` 실행
5. `sbx env plan`으로 변경 사항 확인
6. `sbx env create` 또는 `sbx env run`
7. 필요하면 Sandbox 내부 JDK/SSH/MariaDB 도구 준비
8. Spring Boot 프로젝트 생성/복사
9. MariaDB DB/사용자 초기화
10. `./gradlew clean build`
11. `./gradlew bootRun`
12. `sbx ports wiz-full-dev` 확인
13. LAN PC에서 2222/8080/3307 테스트

32. 매일 개발할 때의 정석 순서

```bash
cd wiz-sandbox

sbx env run --env-arg profile=dev

# 별도 터미널
sbx env exec -- ./gradlew clean build

# 테스트
sbx env exec -- ./gradlew test

# 실행
sbx env exec -- ./gradlew bootRun
```

33. Codex 중심 개발 루프

```text
1. Codex에게 요구사항 설명
2. Codex가 프로젝트 분석
3. Kotlin/Spring Boot 코드 수정
4. ./gradlew clean build
5. 테스트 실패 분석
6. 코드 재수정
7. ./gradlew test
8. Spring Boot 실행
9. REST API curl 테스트
10. git diff 검토
```

34. 상태 확인 명령

```bash
sbx version
sbx ls
sbx ports wiz-full-dev
sbx policy inspect wiz-full-dev

sbx env exec -- java -version
sbx env exec -- ./gradlew --version
sbx env exec -- bash -lc "ps aux"
sbx env exec -- bash -lc "ss -lntp"
```

35. SSH가 안 될 때

확인	명령/조치
Host 포트	Test-NetConnection HOST_IP -Port 2222
포트 게시	sbx ports wiz-full-dev
Sandbox 22 listen	sbx env exec -- bash -lc "ss -lntp | grep ':22 ' || true"
sshd 프로세스	sbx env exec -- bash -lc "ps aux | grep sshd"
사용자	Sandbox 내부 계정 존재 확인
인증	authorized_keys 또는 비밀번호 설정 확인

확인	명령/조치
방화벽	Docker Host OS 방화벽 2222 허용

36. Spring Boot 8080이 안 될 때

```bash
sbx env exec -- ./gradlew bootRun
sbx env exec -- bash -lc "ss -lntp | grep 8080 || true"
sbx env exec -- curl -v http://127.0.0.1:8080/
sbx ports wiz-full-dev
```

내부 curl은 되는데 외부에서 안 되면 port publish/host firewall을 우선 확인함. 내부 curl부터 안 되면 Spring Boot 기동 로그와 애플리케이션 오류를 먼저 확인함.

37. MariaDB 3307이 안 될 때

```bash
sbx env exec -- bash -lc "ss -lntp | grep 3306 || true"
sbx env exec -- mariadb -h 127.0.0.1 -P 3306 -u devuser -p
sbx ports wiz-full-dev
```

Access denied는 네트워크보다 사용자/비밀번호/host 권한 문제일 가능성이 큼. Connection refused는 MariaDB 미기동 또는 listen 주소/포트 문제를 우선 확인함.

38. Gradle build가 안 될 때

```bash
sbx env exec -- java -version
sbx env exec -- ./gradlew --version
sbx env exec -- ./gradlew clean build --stacktrace
```

- JDK 버전 확인
- gradlew 실행권한 확인
- 의존성 다운로드 네트워크 확인
- Kotlin/Spring Boot plugin 버전 확인
- MariaDB 연결이 테스트 단계에서 필요한지 확인

39. Codex 인증이 안 될 때

```bash
sbx secret ls
sbx secret set openai --oauth
sbx env rm
sbx env run
```

환경의 secrets/bindings 등 생성 시 적용되는 항목은 기존 Sandbox에 즉시 반영되지 않을 수 있으므로 환경 재생성이 필요한 경우가 있음.

40. 환경 변경 후 재생성해야 하는 경우

공식 환경 파일 동작상 workspace, kits, ports, secrets, bindings, sandboxOptions 등의 변경은 기존 Sandbox에 즉시 적용되지 않고 다음 생성 시 반영되는 항목이 있음. 이런 경우:

```bash
sbx env plan
sbx env rm
sbx env create
sbx env run
```

41. 보안 권장 구성

- Spring Boot 8080만 LAN 공개하고 SSH/MariaDB는 가능하면 loopback 유지.
- SSH는 비밀번호보다 public key 인증을 우선 고려.
- MariaDB 3307을 인터넷 공유기 포트포워딩으로 직접 공개하지 않음.
- OpenAI API 키를 sbxenv.yaml, application.yml, Git 저장소에 기록하지 않음.
- DB 비밀번호도 Git에 넣지 않음.
- docs read-only workspace를 사용해 중요한 원본 자료의 우발적 수정을 방지.
- 삭제 전 workspace 백업과 MariaDB dump를 별도로 수행.

42. 더 안전한 포트 설정 예

```yaml
ports:
  # SSH는 Docker Host에서만
  - sandbox: 22
    host: 2222
    hostIP: 127.0.0.1
    protocol: tcp4

  # Spring Boot만 LAN 공개
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4

  # MariaDB도 Host에서만
  - sandbox: 3306
    host: 3307
    hostIP: 127.0.0.1
    protocol: tcp4
```

실제 개발에서는 이 구성이 전체 0.0.0.0 공개보다 더 권장됨.

43. 완전체 실행 명령 모음

```bash
# 1. OpenAI 인증
sbx secret set openai --oauth

# 2. 환경 검토
sbx env plan

# 3. 개발 profile로 실행
sbx env run --env-arg profile=dev

# 4. Bash
sbx env exec -it -- bash

# 5. Java/Gradle
sbx env exec -- java -version
sbx env exec -- ./gradlew --version

# 6. Build
sbx env exec -- ./gradlew clean build

# 7. Test
sbx env exec -- ./gradlew test

# 8. Spring Boot
sbx env exec -- ./gradlew bootRun

# 9. 포트
sbx ports wiz-full-dev

# 10. 목록
sbx ls

# 11. 제거 전 계획
sbx env plan

# 12. 제거
sbx env rm
```

44. 초보자가 기억할 핵심 10개

No.	명령	의미
1	sbx env plan	YAML 적용 전에 반드시 확인
2	sbx env run	환경 적용 + Codex attach
3	sbx env exec -it -- bash	Sandbox shell
4	sbx env exec -- ./gradlew clean build	Spring Boot 빌드
5	sbx ports wiz-full-dev	포트 확인
6	sbx ls	Sandbox 목록
7	sbx secret set openai --oauth	Codex 인증
8	sbx env rm	환경 제거

No.	명령	의미
9	Test-NetConnection HOST -Port 8080	Windows 외부 포트 검사
10	ss -lntp	Sandbox 내부 listen 검사

45. Docker Compose와 sbxenv.yaml 비교

구분	docker-compose.yml	sbxenv.yaml
주 목적	여러 컨테이너 서비스 오케스트레이션	AI agent Sandbox 환경 선언
MariaDB service	services로 별도 컨테이너 선언 가능	Compose services 방식 아님
Agent	기본 개념 아님	agent: codex 등
Workspace	volume/bind mount	workspace/additionalWorkspaces
Port	ports	ports
CPU/RAM	deploy/resource 등 구성	sandboxOptions
Secret	Compose secret/env 등	Sandbox service secret/credentials
Lifecycle	entrypoint/command 등	호스트 lifecycle command

MariaDB를 Spring Boot와 완전히 분리된 별도 컨테이너로 안정적으로 운영하고 싶다면 Docker Compose가 더 자연스러운 구조일 수 있음. Docker Sandboxes는 Codex 같은 agent를 격리된 개발환경에서 작업시키는 데 초점이 있음.

46. 최종 권장 운영 방식

학습/개발 단계에서는 Docker Sandbox 안에 개발 도구와 MariaDB를 함께 두는 구성이 이해하기 쉬움. 그러나 장기 운영에서는 다음처럼 역할을 분리하는 것이 관리하기 쉬움.

```text
Docker Sandbox
├── Codex AI
├── JDK 21
├── Kotlin
├── Gradle
└── Spring Boot 개발/테스트

별도 DB 계층
└── MariaDB
    ├── Docker Compose 또는 전용 DB 서버
    ├── 독립 백업
    └── 접근제어
```

즉, Sandbox는 'AI 개발 워크스테이션', MariaDB는 '영구 데이터 서비스'로 분리하는 구조가 장기적으로 더 명확함. 다만 본 교본은 올인원 학습형 구성도 충분히 이해할 수 있도록 함께 설명함.

47. 공식 문서 기준 주의사항

- Docker Sandboxes 0.43.0은 2026-09-15 릴리스 기준이며 기능이 빠르게 변경되고 있음.
- `sbx env`와 `sbxenv.yaml`은 experimental 기능임.
- 프로젝트 환경 파일 이름은 현재 `sbxenv.yaml`이며 사용자 기본값은 `~/.sbxenv.yaml`임.

- 환경 plan은 host lifecycle command, credentials, bindings, MCP, workspace, kit, ports, sandbox options 등을 보여줌.
- workspace/kits/ports/secrets/bindings/sandboxOptions 변경은 기존 Sandbox를 재생성해야 반영되는 경우가 있음.
- Cloud persistent volume은 cloud-only이고 종료 시 snapshot 방식임.

48. 공식 참고자료

- Docker Docs - Docker Sandboxes release notes
- Docker Docs - Sandbox environment files
- Docker Docs - sbx CLI reference
- Docker Docs - sbx ports
- Docker Docs - sbx secret
- Docker Docs - sbx volume

49. 마지막 체크리스트

☐ sbxenv.yaml이 workspace 밖에 있는가?

☐ Codex agent가 지정되어 있는가?

☐ CPU 4 / RAM 8GB가 필요한가?

☐ OpenAI Secret을 평문으로 넣지 않았는가?

☐ SSH 2222 포트가 정말 LAN 공개가 필요한가?

☐ Spring Boot 8080 listen 상태가 정상인가?

☐ MariaDB 3306 listen 상태가 정상인가?

☐ Host 3307 포트를 인터넷에 공개하지 않았는가?

☐ workspace가 호스트에 실제로 존재하는가?

☐ docs가 readOnly로 연결되어 있는가?

☐ 삭제 전 backup이 만들어지는가?

☐ DB는 mysqldump 등 별도 백업이 있는가?

☐ 환경 변경 후 Sandbox 재생성이 필요한지 확인했는가?